PPS - Scala Stream Fusion

Leonardo Naddei

26 ottobre 2026

Indice

1 Introduzione **3**

2 Analisi dei requisiti **4**
2.1 Requisiti di business . . . 4
2.2 Requisiti funzionali utente . . . 4
2.3 Requisiti funzionali di sistema . . . 5
2.4 Requisiti negativi ed esclusioni (Out of Scope) . . . 5
2.5 Requisiti non funzionali . . . 6

3 Background e tecnologie utilizzate **7**
3.1 Stream fusion . . . 7
3.1.1 Push stream . . . 7
3.1.2 Pull stream . . . 8
3.2 Metaprogrammazione in Scala 3 . . . 8
3.2.1 Introduzione alla Multi-stage programming in Scala 3 . . . 8
3.3 Metaprogrammazione analitica . . . 9
3.3.1 La reflection API e l'AST tipizzato . . . 10
3.3.2 Tipi staged e tipi riflessivi . . . 10
3.3.3 Beta reduction delle funzioni . . . 11
3.4 Definizione di variabili . . . 11

4 Design architetturale **12**
4.1 Architettura generale . . . 12
4.2 Componenti del sistema . . . 12
4.3 Architettura del compilatore . . . 13

5 Design di dettaglio **14**
5.1 Design dell'API pubblica . . . 14
5.1.1 Operazioni terminali . . . 14
5.1.2 Tipologie di stream . . . 15
5.1.3 Vincoli negativi sui tipi . . . 15
5.1.4 Operazione terminale `collect` . . . 16
5.1.5 Configurazione compile-time e runtime . . . 17
5.1.6 Import semplificato dell'API . . . 17
5.2 Design della Stream IR . . . 18
5.2.1 Ast: phased indexed container . . . 19
5.3 Rappresentazione per la Code Generation (OP Ir) . . . 20
5.3.1 Valori . . . 21
5.3.2 Operazioni . . . 21
5.4 La collection strategy . . . 22

6 Implementazione **23**
6.1 Parsing . . . 23
6.1.1 Parsing della stream . . . 23
6.1.2 Parsing del collector . . . 24
6.1.3 Parsing della configurazione statica . . . 25
6.2 Enrichment . . . 25
6.2.1 Materializzazione delle espressioni . . . 26
6.2.2 Stato e condizioni di terminazione . . . 26
6.2.3 Analisi della cardinalità . . . 27
6.3 Planning . . . 27
6.3.1 Selezione della strategia terminale . . . 27
6.3.2 Generazione della pipeline e delle sorgenti . . . 28
6.3.3 Generazione del piano parallelo . . . 29
6.4 Lowering . . . 29
6.4.1 Traduzione dell'OP IR . . . 29
6.4.2 Applicazione e inlining delle funzioni . . . 30
6.4.3 Chiamate e inlining dei collector . . . 30
6.4.4 Lowering di pipelines parallele . . . 30
6.5 Esempio di traduzione end-to-end . . . 31
6.5.1 L'input alla libreria . . . 31
6.5.2 Il risultato del parsing . . . 33
6.5.3 L'output dell'enrichment . . . 33
6.5.4 Pianificazione in OP IR . . . 34
6.5.5 Lowering verso l'AST Scala . . . 36
6.5.6 Il bytecode JVM generato . . . 36

7 Testing e valutazione delle prestazioni **38**
7.1 Testing . . . 38
7.2 Valutazione delle prestazioni . . . 39
7.2.1 Metodologia . . . 39
7.2.2 Benchmark sequenziali . . . 39
7.2.3 Benchmark paralleli . . . 40
7.2.4 Livelli di compilazione JIT . . . 40
7.2.5 Allocazioni . . . 41

8 Processo di sviluppo **42**
8.1 Modello di sviluppo . . . 42
8.2 Fasi di sviluppo . . . 42
8.3 Gestione del repository . . . 43
8.4 Automazione . . . 43

9 Conclusioni **44**
9.1 Retrospettiva . . . 44
9.1.1 Difficoltà incontrate . . . 44
9.2 Limiti e compromessi progettuali . . . 44
9.3 Uso degli LLM . . . 45
9.4 Sviluppi futuri . . . 45

Capitolo 1

Introduzione

Il modello di programmazione basato su pipeline dichiarative è sempre più diffuso per l'elaborazione di collezioni e sequenze di dati. Operazioni come *map*, *filter* e *flatMap* permettono di descrivere una computazione separando la logica dell'elaborazione dai dettagli relativi all'attraversamento della struttura dati.

Questa forma di astrazione migliora la leggibilità e la composizionalità del codice, ma può introdurre un costo rispetto alla tipica implementazione imperativa. In particolare, sulla JVM, una pipeline ad alto livello può richiedere la costruzione di strutture intermedie, l'allocazione di capturing lambda, l'invocazione indiretta delle funzioni associate agli operatori e, in alcuni casi, operazioni di boxing e unboxing evitabili. Il JIT compiler può eliminare parte di questi costi, ma la sua efficacia dipende dalla struttura del programma.

Il problema è noto anche in sistemi reali. Nel progetto Apache Cassandra, ad esempio, sono state introdotte ottimizzazioni mirate alla rimozione delle Java Stream da percorsi di esecuzione particolarmente frequenti [4] e all'eliminazione di capturing lambda [3]. Questi interventi mostrano come possa risultare necessario rinunciare ad astrazioni di alto livello per ottenere un'esecuzione più efficiente.

Il limite dei sistemi basati sull'applicazione dinamica di lambda function emerge, soprattutto, quando lo stesso call site viene utilizzato con più lambda differenti. In questi casi il call site diventa *megamorfico*. Il problema è noto e sono state proposte diverse strategie per preservare e propagare maggiori informazioni di specializzazione. Tra queste, la proposta della *Parametric VM* discute meccanismi per associare informazioni di specializzazione a metodi e call site, consentendo alla JVM di generare codice maggiormente specializzato [7]. La libreria evita invece di affrontare questo problema a runtime: quando la funzione è nota staticamente, ne effettua la monomorfizzazione a e l'inlining a compile-time.

Il problema affrontato in questo progetto consiste quindi nel ridurre il divario tra due modalità di esprimere la stessa computazione: da un lato una pipeline dichiarativa e composizionale, dall'altro un ciclo imperativo esplicitamente specializzato per l'elaborazione da eseguire. L'obiettivo è quindi preservare il primo modello di programmazione producendo codice vicino al secondo.

Il principio alla base della trasformazione è quello della *stream fusion*, introdotto da Coutts et al. in *Stream Fusion: From Lists to Streams to Nothing at All* [1]: le operazioni intermedie vengono fuse in un singolo flusso di esecuzione, evitando la materializzazione dei risultati intermedi.

Particolare attenzione è stata dedicata alla progettazione dell'API. Le proprietà e i vincoli della pipeline vengono rappresentati attraverso il type system. La libreria è inoltre estendibile da parte degli utilizzatori per quanto riguarda le operazioni terminali.

Capitolo 2

Analisi dei requisiti

2.1 Requisiti di business

RB-1 **Colmare il divario tra espressività e prestazioni.** disponibili nativamente possono essere estese tramite interfacce dedicate. La programmazione dichiarativa tramite pipeline favorisce la leggibilità, l'astrazione e la manutenibilità. Tuttavia, sulla JVM tali astrazioni comportano comunemente una penalizzazione prestazionale. L'obiettivo primario del progetto è annullare questo compromesso, consentendo di scrivere codice ad alto livello che compili direttamente in codice efficiente.

RB-2 **Adozione a basso attrito.** L'integrazione della libreria non deve richiedere modifiche alla toolchain standard del compilatore né l'apprendimento di API complesse. La sintassi offerta deve risultare familiare a chi già utilizza le collezioni di Scala o le API Java Stream, garantendo un'interfaccia compatta, uniforme e accessibile tramite un singolo import.

2.2 Requisiti funzionali utente

RFU-1 **Creazione dello Stream.** L'utente deve poter effettuare elaborazioni a partire da diverse sorgenti di dati (ad esempio collezioni Scala, array o iteratori) attraverso un unico punto di ingresso.

RFU-2 **Configurazione.** L'utente deve poter configurare il comportamento del sistema, sia per quanto riguarda i parametri di generazione statica del codice che per i parametri riguardanti l'esecuzione a runtime. In assenza di configurazione esplicita, il sistema deve utilizzare valori predefiniti senza richiedere ulteriori interventi da parte dell'utente.

RFU-3 **Esecuzione parallela esplicita.** Qualora la sorgente dei dati sia adatta, l'utente deve poter richiedere l'esecuzione parallela della pipeline.

RFU-4 **Raccolta dei risultati.** L'utente deve poter determinare il risultato tramite funzioni predefinite della libreria (come ad esempio collezionando in lista) ma anche in maniera personalizzata tramite interfacce specifiche.

RFU-5 **Guida contestuale tramite Type System.** L'API deve guidare attivamente l'utente durante la costruzione della pipeline sfruttando il type system. Tramite tipi e subtyping avanzato, l'IDE e il compilatore devono esporre ed accettare solo ed esclusivamente le operazioni valide per lo stato corrente dello stream (es. inibendo metodi non supportati in parallelo).

RFU-6 **Comportamento prevedibile dell'API.** Le operazioni esposte dalla libreria devono seguire semantiche coerenti con quelle comunemente associate alle pipeline di elaborazione di collezioni, minimizzando comportamenti inattesi per l'utente (*Least Astonishment*). Operazioni equivalenti a quelle offerte dalle API standard devono, ove applicabile, preservarne il significato e le aspettative d'uso.

2.3 Requisiti funzionali di sistema

RFS-1 **Riconoscimento e analisi della pipeline statica.** Il sistema deve essere in grado di ispezionare a compile time l'albero sintattico di una pipeline dichiarativa e scomporla nella sequenza logica di sorgente, trasformazioni intermedie e operazione terminale.

RFS-2 **Fusione delle operazioni sequenziali.** Il sistema deve collassare le trasformazioni in un unico flusso di esecuzione continuo, eliminando la materializzazione di collezioni o iteratori tra uno stadio e il successivo.

RFS-3 **Generazione di codice specializzato.** Il sistema deve emettere codice specializzato per i tipi di dato coinvolti e per l'operazione terminale richiesta, evitando chiamate polimorfiche o indirette.

RFS-4 **Gestione del flusso e terminazione anticipata.** Il sistema deve regolare il flusso di controllo del ciclo generato per supportare sia l'elaborazione esaustiva dell'intero flusso sia l'interruzione anticipata quando la logica dell'operazione terminale o di filtro lo richiede.

RFS-5 **Composizione di flussi annidati.** Il sistema deve consentire la fusione di flussi generati dinamicamente da elementi intermedi (`flatMap`), integrando il ciclo interno nel contesto del ciclo esterno senza creare buffer di accumulazione intermedi o computazioni superflue.

RFS-6 **Supporto all'esecuzione parallela.** Per le sorgenti e i terminali compatibili, il sistema deve emettere codice che esegua la computazione in maniera parallela.

2.4 Requisiti negativi ed esclusioni (Out of Scope)

Al fine di garantire la predicibilità prestazionale e la robustezza del compilatore, sono definiti i seguenti punti come out of scope:

RN-1 **Assenza di composizione dinamica runtime.** Il sistema non deve supportare la costruzione dinamica di pipeline determinate da rami condizionali ordinari a runtime (ad esempio `if (cond) s.map(...) else s.filter(...)`). L'intera catena di trasformazioni deve essere fissa e nota sintatticamente al call-site di compilazione.

RN-2 **Nessuna ottimizzazione opaca sul codice utente.** Il sistema non deve alterare né riscrivere la logica interna delle espressioni arbitrarie fornite dall'utente all'interno delle lambda.

RN-3 **Nessun supporto a flussi annidati di stream.** Il sistema non supporta la generazione di stream i cui singoli elementi siano a loro volta istanze di stream tramite trasformazioni mapping (es. `map(x => FusedStream.of(x))`). La sola composizione nested consentita è quella finalizzata all'appiattimento esplicito tramite `flatMap`.

RN-4 **Errori parlanti e assenza di fall-back.** Se una pipeline viola i vincoli del framework il compilatore deve rigettare esplicitamente il programma tramite messaggi d'errore posizionati e descrittivi, senza conversioni nascoste.

RN-5 **Nessun inlining o ottimizzazione per funzioni non-lambda.** Qualora a una trasformazione (es. `map`) venga passata una funzione preesistente anziché un'espressione lambda, il compilatore non applica la β-reduction né l'inlining statico.

2.5 Requisiti non funzionali

RNF-1 **Prestazioni a runtime.** Il codice generato deve esibire un throughput comparabile a codice efficiente scritto manualmente in Java, riducendo l'overhead computazionale rispetto alle pipeline funzionali convenzionali. Per sorgenti a oggetti generici invece il boxing è legato al comportamento standard della JVM e non è evitabile.

RNF-2 **Assenza di allocazioni intermedie.** L'elaborazione delle operazioni intermedie stateless non deve allocare oggetti sull'heap.

RNF-3 **Trasparenza semantica.** A parità di sorgente, funzioni utente e configurazione, il codice generato deve produrre risultati semanticamente equivalenti a quelli definiti dalla pipeline dichiarativa.

Capitolo 3

Background e tecnologie utilizzate

Questo capitolo introduce i principali concetti teorici e gli strumenti di Scala 3 necessari alla comprensione delle scelte progettuali e implementative adottate nel progetto.

3.1 Stream fusion

Una stream pipeline può essere rappresentata secondo due modelli di esecuzione fondamentali: un modello *push*, guidato dal produttore, e un modello *pull*, guidato dal consumatore. Questa distinzione viene discussa, nel contesto della stream fusion, da Kiselyov et al. in *Stream Fusion, to Completeness* [2].

3.1.1 Push stream

Nel modello *push* è la sorgente a guidare l'elaborazione: ogni elemento prodotto viene immediatamente propagato attraverso gli operatori successivi della pipeline fino a raggiungere il terminale. Concettualmente, l'intera sequenza di operatori può quindi essere vista come una "mega-funzione" applicata a ciascun elemento della sorgente.

Per esempio:

```
stream
  .filter(p)
  .map(f)
  .collect(collector)
```

può essere interpretata come una singola funzione data dalla composizione:

$$\text{consume}(x) = \text{collector}\big(\text{map}(\text{filter}(x))\big).$$

Questa rappresentazione rende particolarmente semplice l'implementazione di operatori come `map` e `filter`: ciascun operatore può scartare oppure inoltrare (eventualmente trasformandolo) l'elemento ricevuto.

È più complessa la gestione di operatori che modificano la cardinalità della stream: operazioni come `skip` e `limit` richiedono infatti di mantenere uno stato relativo al numero di elementi osservati. Nel caso di `limit`, una volta raggiunto il numero di elementi, è necessario interrompere anticipatamente la produzione della sorgente.

Una difficoltà ancora maggiore emerge con operatori che combinano più stream come lo zip. Nel modello push ciascuna sorgente possiede infatti un proprio flusso di produzione: per ogni elemento che viene emesso dalla sorgente si può avere un numero diverso di elementi prodotti nel punto di zipping. Per associare l'elemento corrente di una stream con quello corrispondente di un'altra è necessario quindi introdurre un meccanismo di sincronizzazione tra i due produttori.

3.1.2 Pull stream

Nel modello *pull*, al contrario, il controllo dell'esecuzione è affidato al consumatore. Lo stream viene rappresentata come un oggetto capace di produrre, su richiesta, l'elemento successivo. In questo caso non è la sorgente a inviare gli elementi verso gli operatori successivi, ma è il terminale a richiederli risalendo progressivamente la pipeline. Ogni operatore intermedio definisce quindi il proprio comportamento in termini dell'operatore precedente: una `map`, ad esempio, richiede un elemento alla stream sottostante e applica ad esso la funzione di trasformazione. Un `filter` continua invece a richiedere elementi finché non ne incontra uno che soddisfa il predicato.

Questa organizzazione rende naturali operazioni come `limit`, poiché il consumatore può semplicemente cessare di richiedere nuovi elementi, e soprattutto operatori come *zip*: è sufficiente richiedere contemporaneamente un elemento a ciascuna delle due stream e combinarne i risultati. Il prezzo da pagare è una rappresentazione esplicita dello stato dell'iterazione e, in assenza di successive ottimizzazioni, un maggior numero di passaggi intermedi necessari alla produzione di ciascun elemento.

3.2 Metaprogrammazione in Scala 3

Il progetto richiede sia la generazione di nuovo codice sia l'analisi e la trasformazione del codice ricevuto dalla macro. Questa sezione introduce i meccanismi di metaprogrammazione di Scala 3 utilizzati a tale scopo. Quanto segue e gli esempi riportati fanno riferimento al modello di metaprogrammazione generativa e analitica presentati in [8, 9].

3.2.1 Introduzione alla Multi-stage programming in Scala 3

Le due primitive fondamentali del sistema di metaprogrammazione staged di Scala 3 sono le *quote* e gli *splice*.

Una quote `'{ expression }` costruisce una rappresentazione del codice contenuto al suo interno. Se `expression` produce un valore di tipo `T`, il risultato della quote ha tipo `Expr[T]` e rappresenta codice che produrrà un valore di tipo `T`.

Uno splice `${expression}` opera invece nella direzione opposta e inserisce all'interno del codice quotato il frammento di programma rappresentato da un'espressione `Expr[T]`.

Un esempio di generazione staged è l'elevamento a potenza con esponente noto a compile time:

```
def powCode(x: Expr[Int], n: Int)(using Quotes): Expr[Int] =
  if n == 0 then
    Expr(1)
  else
    '{ $x * ${ powCode(x, n - 1) } }
```

L'espressione

```
'{ $x * ${ powCode(x, n - 1) } }
```

combina codice generato e computazioni eseguite durante l'espansione della macro. In particolare:

- `'{' ... '}'` è la *quote* esterna: costruisce un nuovo valore `Expr[Int]` che rappresenta l'espressione di moltiplicazione generata.
- `x` è un parametro di tipo `Expr[Int]` e rappresenta quindi un frammento di codice, non direttamente un valore intero. Lo splice `$x` inserisce tale frammento come operando della moltiplicazione.

- `powCode(x, n - 1)` è invece una computazione eseguita ricorsivamente durante l'espansione della macro
- lo splice `${ powCode(x, n - 1) }` inserisce il codice prodotto dalla chiamata ricorsiva come operando destro della moltiplicazione
- l'operatore `*` appartiene infine al codice contenuto nella quote: non viene eseguito durante la generazione, ma diventa parte del programma generato

La chiamata a `powCode` e la sua ricorsione avvengono a *compile time*, le moltiplicazioni costruite all'interno delle quote saranno eseguite a *runtime*.

Ad esempio, per `n = 3`, le successive espansioni possono essere viste come:

```
powCode(x, 3)
=> '{ $x * ${powCode(x, 2)} }
=> '{ $x * $x * ${powCode(x, 1)} }
=> '{ $x * $x * $x * ${powCode(x, 0)} }
=> '{ $x * $x * $x * 1 }
```

3.3 Metaprogrammazione analitica

Una macro generativa costruisce un nuovo frammento di programma a partire da informazioni note durante la compilazione, come nell'esempio di `powCode` appena visto. È tuttavia necessario anche compiere l'operazione inversa: ricevere un frammento di programma e analizzarne la struttura.

Scala permette di farlo in maniera elementare attraverso il pattern matching su codice quoted. Un'espressione può essere analizzata come segue:

```
expr match {
case '{ someFunction($argument) } =>
...
}
```

Il pattern quoted descrive la struttura sintattica che si intende riconoscere. Le parti precedute da `$` costituiscono invece i pattern di frammenti presenti nelle corrispondenti posizioni dell'espressione da catturare.

Ad esempio, un'addizione tra due espressioni intere:

```
def inspect(expr: Expr[Int])(using Quotes): Unit = expr match
    case '{ ($left: Int) + ($right: Int) } => ...
    case _ =>  println("Not recognized")
```

Applicando `inspect` all'espressione `'{ x + 42 }` il pattern riconosce l'operazione di addizione e associa a `left` e `right` i due frammenti:

$$\texttt{left} \longrightarrow \texttt{'\{x\}} \qquad \texttt{right} \longrightarrow \texttt{'\{42\}}.$$

Entrambe le variabili hanno tipo `Expr[Int]`: non contengono i valori ma rappresentano il codice corrispondente.

Quote e splice assumono quindi un ruolo complementare anche nell'analisi: all'interno dei quoted patterns, la quote descrive la forma del codice da riconoscere, mentre gli splice identificano le porzioni variabili della struttura e le rendono disponibili come valori `Expr[T]` per le successive fasi della macro.

Scala offre anche una modalità alternativa e più generale per l'analisi del codice: la *Reflection API*, che consente di accedere direttamente alla rappresentazione dell'albero sintattico prodotto dal compilatore.

3.3.1 La reflection API e l'AST tipizzato

Quando è necessario analizzare direttamente il programma è possibile convertire una `Expr[T]` nel relativo nodo dell'AST del compilatore con `expr.asTerm`. La Reflection API espone la struttura prodotta dal compilatore, i nodi più importanti utilizzati nel parser sono:

- `Apply`, per l'applicazione di una funzione
- `TypeApply`, per un'applicazione contenente parametri di tipo
- `Select`, per la selezione di un membro
- `Ident`, per il riferimento a un identificatore
- `Block`, per un blocco contenente dichiarazioni e un'espressione finale
- `Inlined`, per rappresentare il risultato di un'espansione inline
- `ValDef`, `Assign` e `Ref`, utilizzati durante la generazione di nuovo codice

Ad esempio, l'espressione `numbers.map(f)` può essere rappresentata, in forma semplificata, dal seguente albero:

```
Apply(
  Select(Ident("numbers"), "map"),
  List(Ident("f"))
)
```

Tale struttura può essere analizzata direttamente mediante pattern matching:

```
expr.asTerm match
  case Apply(Select(source, "map"), List(function)) => ...
```

Il pattern permette quindi di riconoscere l'operazione `map` e di estrarre separatamente il termine `source`, che rappresenta la sorgente della chiamata, e `function`, che ne rappresenta l'argomento. L'analisi programmatica dell'AST consente quindi di riconoscere e manipolare strutture più complesse e variabili rispetto al quoted pattern matching, senza dover descrivere a priori l'intera forma sintattica dell'espressione mediante un singolo pattern.

3.3.2 Tipi staged e tipi riflessivi

Scala mette a disposizione due rappresentazioni dei tipi:

`Type[T]` rappresenta il tipo `T` nell'API staged ed è utilizzabile insieme a `Expr[T]`, quote e splice;

`TypeRepr` rappresenta invece un tipo all'interno della Reflection API e permette di ispezionarne direttamente la struttura.

Il passaggio dalla rappresentazione riflessiva a quella staged avviene mediante `asType`. Data una `TypeRepr`, questo metodo restituisce un `Type[?]`, sul quale è possibile effettuare pattern matching per rendere disponibile il tipo sottostante:

```
item.asType match {
  case '[t] => ...
}
```

Il pattern `'[t]` introduce `t` come tipo utilizzabile all'interno del relativo ramo. Questo permette, ad esempio, di partire da un termine dell'AST di tipo non noto a priori e generare nuovo codice che ne preservi il tipo:

```
def makeList(term: Term)(using Quotes): Expr[Any] = term.tpe.asType match
    case '[t] =>
      val value: Expr[t] = term.asExprOf[t]
      '{ List[t]($value) }
```

Dato il tipo di `term`, `term.tpe` ne fornisce la rappresentazione riflessiva `TypeRepr`. Il pattern `'[t]` rende tale tipo disponibile come `t`. A questo punto `t` può essere utilizzato all'interno di una quote per costruire nuovo codice tipizzato.

Questo meccanismo fa da collegamento tra Reflection API e staged programming: un tipo individuato durante l'analisi dell'AST può essere portato nel contesto staged e utilizzato per costruire nuove espressioni preservandone l'informazione di tipo.

3.3.3 Beta reduction delle funzioni

Un altro elemento importante è la funzionalità di β-reduction. Data una funzione:

$$f = x \mapsto e, \tag{3.1}$$

`f(a)` può essere calcolata per sostituzione, sostituendo direttamente x con a all'interno di e. Scala rende disponibile questa trasformazione attraverso: `Expr.betaReduce(...)`. Incorporando staticamente il corpo della lambda nell'IR generato, la macro elimina sia la rappresentazione della funzione come entità invocabile sia il dispatch dell'invocazione.

3.4 Definizione di variabili

Nell'AST di Scala, un riferimento a una variabile è associato a un `Symbol`. Il nome è solamente una proprietà del simbolo e non ne rappresenta l'identità. La Reflection API permette di creare nuove variabili attraverso: `Symbol.newVal(...)`, e di fare successivamente riferimento allo stesso simbolo attraverso `Ref(symbol)`. La definizione viene creata nell'AST tramite `ValDef(symbol, Some(initializer))`. La distinzione tra nome e simbolo garantisce inoltre l'*hygiene* della macro: variabili generate dalla macro non possono andare in conflitto né con variabili omonime definite nel codice utente né con altre variabili create durante l'espansione della macro.

Capitolo 4

Design architetturale

4.1 Architettura generale

Il progetto è organizzato in due aree principali: l'*API pubblica* e il *compilatore*.

L'API pubblica, descritta nella Sezione 5.1, espone il modello con cui l'utente definisce la pipeline. L'invocazione di un'operazione terminale attiva il compilatore, che analizza staticamente la pipeline e genera il codice necessario alla sua esecuzione.

4.2 Componenti del sistema

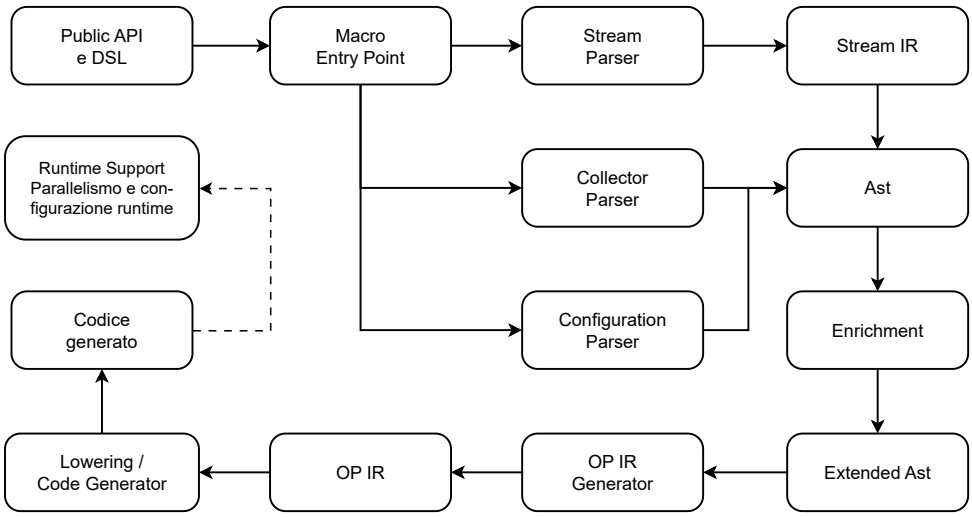


Figura 4.1: Architettura generale e principali componenti del sistema. Le frecce continue rappresentano il flusso di compilazione della pipeline, le frecce tratteggiate indicano le dipendenze a runtime.

Come mostrato in Figura 4.1, il flusso di compilazione parte dalla Public API e termina con la produzione del codice eseguibile. Il *Macro Entry Point* costituisce il punto di ingresso del compilatore e orchestra le diverse fasi di trasformazione.

Il *Parser* analizza l'AST Scala della pipeline e ne ricostruisce la rappresentazione logica nella *Stream IR*. Analogamente anche il collector e la configurazione vengono analizzate per determinare la strategia terminale e i parametri che influenzano la generazione e l'esecuzione del codice. Il risultato del parsing della pipeline e del collector è un `Ast(Raw)`, che raccoglie l'albero, la strategia terminale, le istruzioni preliminari e la modalità di esecuzione.

L'AST viene successivamente analizzato ed esteso dalla fase di *Enrichment*, che associa alla pipeline le informazioni statiche necessarie alla pianificazione dell'esecuzione, tra cui proprietà

della sorgente, cardinalità e condizioni di terminazione. Questa fase produce un `Ast(Enriched)`. Il *Planner* utilizza questa rappresentazione arricchita per produrre la *Operation IR* (OP IR), nella quale le operazioni logiche della pipeline vengono trasformate in un piano orientato all'esecuzione. Infine, il *Lowerer* traduce la OP IR in un nuovo AST Scala, che viene restituito al compilatore e reinserito nel normale processo di compilazione. Il *Runtime Support*, esterno al flusso principale di trasformazione, fornisce invece le funzionalità richieste dal codice generato, in particolare per l'esecuzione parallela e la configurazione runtime.

4.3 Architettura del compilatore

La pipeline interna del compilatore può essere suddivisa concettualmente in *frontend* e *backend*, come mostrato in Figura 4.2.

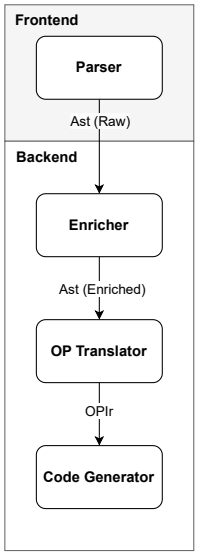


Figura 4.2: Architettura generale del compilatore di FusedStream.

Il *frontend* è responsabile dell'interpretazione della DSL: analizza l'AST Scala ricevuto dalla macro e ricostruisce la struttura della pipeline nella Stream IR. Questa rappresentazione mantiene una corrispondenza diretta con i concetti del dominio, quali sorgenti e operatori intermedi.

Il *backend* trasforma progressivamente questa rappresentazione fino a ottenere il codice eseguibile. L'Enrichment completa la Stream IR con le informazioni statiche necessarie alla pianificazione. Il Planner traduce quindi la pipeline arricchita in una rappresentazione orientata all'esecuzione. Il Lowerer, infine, converte la OP IR nel nuovo AST Scala.

Il flusso complessivo espresso come di rappresentazioni è:

$$\text{Scala AST} \longrightarrow \text{Ast(Raw)} \longrightarrow \text{Ast(Enriched)} \longrightarrow \text{OP IR} \longrightarrow \text{Generated Scala AST}.$$

Capitolo 5

Design di dettaglio

5.1 Design dell'API pubblica

L'obiettivo nel design dell'API pubblica è stato quello di fornire un'interfaccia che garantisse contemporaneamente:

- la possibilità di descrivere pipeline sufficientemente espressive
- l'estendibilità, in particolare per quanto riguarda le operazioni terminali
- il maggior numero possibile di garanzie a compile time
- l'eliminazione di codice che verificasse esplicitamente la correttezza delle combinazioni di operazioni richieste dall'utente eliminando la necessità di documentarli.

Dal punto di vista concettuale, ogni stream può essere scomposto in tre componenti principali:

1. una sorgente, generalmente costituita da una collezione o da un array;
2. una sequenza di operazioni intermedie (`map`, `filter`,`flatMap`,..)
3. un'operazione terminale

L'organizzazione dei tipi esposti dall'API riflette direttamente questa struttura. Tramite sealed trait, tipi parametrici, tipi opachi e vincoli sui parametri di tipo è stato possibile modellare tutte le proprietà della pipeline. In questo modo il compilatore Scala e l'IDE possono guidare l'utente durante la costruzione dello stream e impedire le combinazioni non supportate. L'API mantiene, allo stesso tempo, un certo grado di estendibilità consentendo all'utente di definire nuove operazioni terminali attraverso l'interfaccia `Collector`.

5.1.1 Operazioni terminali

Le operazioni terminali sono rappresentate attraverso l'astrazione `Collector`. La rappresentazione completa è riportata in Figura 5.2.

I terminali sono parametrizzati tramite quattro parametri di tipo che rappresentano rispettivamente:

- `E`: il tipo degli elementi ricevuti dallo stream;
- `B`: il tipo dello stato intermedio, o buffer, utilizzato durante l'accumulazione;
- `R`: il tipo del risultato finale prodotto dal collector;

- `S`: la politica di terminazione del collector.

Il metodo `supplier` crea lo stato iniziale dell'accumulator, `accumulator` aggiunge un nuovo elemento, il valore booleano restituito segnala che l'elaborazione può essere interrotta (si pensi a una findFirst). Infine, `finisher` trasforma lo stato intermedio nel risultato finale.

I collector possono essere esaustivi ovvero richiedere l'elaborazione dell'intera sorgente, oppure possono terminare anticipatamente senza consumare tutti i dati. Questa proprietà è rappresentata tramite `TerminationPolicy`, i cui sottotipi `Exhaustive` e `ShortCircuiting` distinguono rispettivamente i collector che richiedono l'elaborazione completa della sorgente da quelli che possono terminare anticipatamente (Figura 5.2).

Collector combinabili e parallelismo

Per poter eseguire una riduzione in parallelo è necessario ma non sufficiente che il collector sia `Exhaustive`: serve anche una funzione che permetta di combinare gli stati intermedi prodotti indipendentemente dai diversi worker.

Questa proprietà è rappresentata da `Combinable[B]`. Come mostrato in Figura 5.2, `ParallelCollector` combina le due proprietà richieste, modella infatti un `Collector Combinable[B]` con politica `Exhaustive`. Il type system garantisce quindi che una pipeline parallela possa utilizzare soltanto collector adeguati.

Intrinsic vs. first-class collectors

Alcune operazioni terminali ad alta frequenza d'uso, come `toArray` e `summing`, beneficiano di un trattamento specializzato. Come illustrato in Figura 5.2, tali operazioni sono modellate rispettivamente mediante i tipi opachi `ToArrayCollector` e `SummingCollector`, entrambi sottotipi di `ParallelCollector`.

A differenza dei collector convenzionali, queste entità non operano semplicemente come valori gestiti a runtime tramite l'interfaccia `Collector`: la loro presenza lungo la pipeline viene intercettata staticamente dal compilatore, innescando una strategia di code generation dedicata. L'adozione di tipi opachi consente quindi di esporre a compile time proprietà come il supporto all'esecuzione parallela pur mantenendo incapsulati i dettagli implementativi.

Al contrario, soluzioni predefinite quali `toList` e `toSet` sono costrutti first-class: non godono di alcuna conoscenza privilegiata da parte del compilatore, sono presenti all'interno della libreria come pure convenience implementation e gestite al pari di qualsiasi collector definito da un utente.

5.1.2 Tipologie di stream

La modalità di esecuzione della pipeline è rappresentata staticamente attraverso la gerarchia di tipi mostrata in Figura 5.1. `SequentialStream` e `ParallelStream` distinguono rispettivamente le pipeline sequenziali e parallele, mentre `ParallelizableSource` identifica le sorgenti sequenziali per le quali è disponibile la transizione alla modalità parallela.

I sottotipi preservano la modalità di esecuzione lungo la pipeline: ad esempio, una `map` applicata a una `ParallelStream[A]` produce una `ParallelStream[B]`. Analogamente, la disponibilità di alcune operazioni è vincolata dal tipo statico: `skip` e `limit`, definite esclusivamente per `SequentialStream`, non sono disponibili su una `ParallelStream`.

5.1.3 Vincoli negativi sui tipi

Alcune proprietà dell'API sono espresse mediante vincoli negativi tramite `NotGiven` sui parametri di tipo. In particolare si richiede che una determinata evidenza implicita *non* sia disponibile.

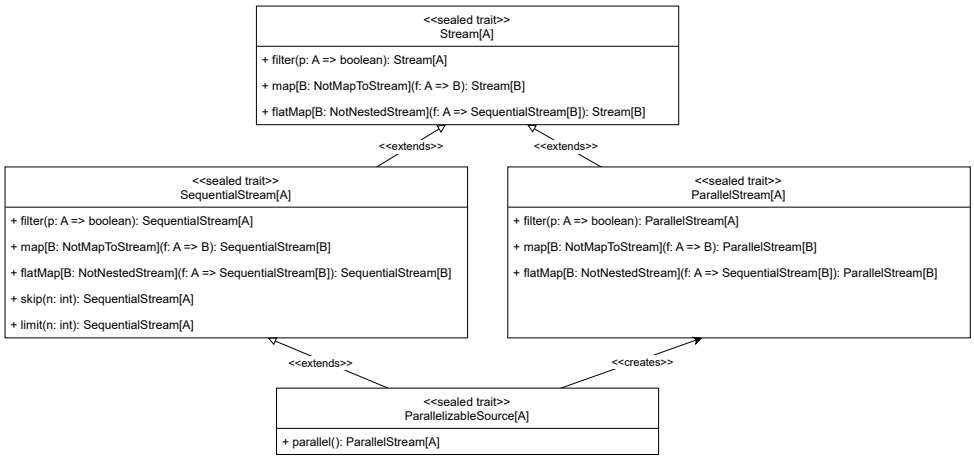


Figura 5.1: Gerarchia dei tipi utilizzati per rappresentare le modalità di esecuzione degli stream.

Questo meccanismo viene utilizzato per impedire la costruzione di stream annidati attraverso operazioni. Ad esempio, il metodo `map` è definito come:

```
def map[B: NotMapToStream](f: A => B): Stream[B]
```

dove l'evidenza `NotMapToStream[B]` può essere costruita soltanto quando `B` non è un sottotipo di `Stream[?]`:

```
given [T](using NotGiven[T <:< Stream[?]]): NotMapToStream[T] = new NotMapToStream[
    T] {}
```

Di conseguenza, un'espressione come `stream.map(x => FusedStream.from(...))` viene rifiutata a compile time.

Tramite le annotazioni `@implicitNotFound` vengono emessi messaggi di errore chiari e specifici. In questo modo combinazioni non supportate vengono escluse direttamente durante la risoluzione dei tipi, senza richiedere controlli espliciti nella successiva fase di trasformazione della pipeline.

5.1.4 Operazione terminale `collect`

L'operazione `collect`, definita tramite extension methods, costituisce il punto nel quale la descrizione dichiarativa della pipeline viene trasformata staticamente nel codice imperativo che ne realizza l'esecuzione.

Una pipeline sequenziale può utilizzare genericamente un `Collector`, mentre una pipeline parallela richiede un `ParallelCollector`, ovvero un collector esaustivo i cui risultati parziali siano combinabili. Questa distinzione permette al type system di rifiutare staticamente combinazioni non supportate:

```
stream.parallel().collect(Collector.findFirst) // rifiutato a compile-time
stream.parallel().collect(Collector.toArray)   // valido
```

`findFirst` è infatti marcato come `ShortCircuiting` e non soddisfa i requisiti di `ParallelCollector`, mentre `ToArrayCollector[T]` ne è un sottotipo esaustivo. La gerarchia completa dei collector e delle relative politiche di terminazione è illustrata in Figura 5.2.

Inlining dei Collector e la modalità `strictInlining`. Per ridurre le allocazioni e il dispatch dinamico a runtime, la libreria effettua l'inlining delle chiamate ai metodi del collector di

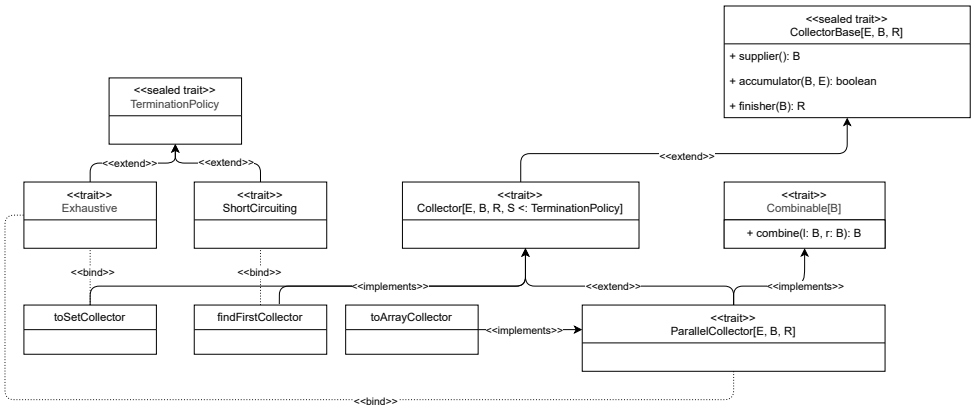


Figura 5.2: Gerarchia dei Collector, politiche di terminazione e vincoli di inlining. La relazione «bind» rappresenta la specializzazione statica di un parametro di tipo (ad es. `S` con `Exhaustive` o `ShortCircuiting`).

default (modalità `strictInlining = true`). In questo caso il collector fornito a `collect` deve essere una classe concreta, rifiutando classi anonime, classi astratte e trait inoltre, le funzioni del collector devono essere obbligatoriamente dichiarate come `inline def`.

Queste condizioni sono verificate a compile time, durante l'esecuzione della macro. Questa modalità può essere disabilitata tramite `strictInlining = false`, in questo caso rimarranno le chiamate dinamiche ai metodi del collector.

5.1.5 Configurazione compile-time e runtime

La libreria distingue tra due tipologie di configurazione:

`CompileConfig` (Configurazione di compilazione). Determina le strategie di manipolazione dell'AST e le ottimizzazioni applicate dalla macro a compile time:

- `useUnsafe`: quando abilitato, sfrutta `VarHandle` per accedere direttamente al vettore interno di `java.util.ArrayList`
- `enableLogging`: abilita il logging dell'esecuzione della macro
- `strictInlining`: abilita l'inlining delle chiamate ai collector

`RuntimeConfig` (Configurazione di esecuzione). Determina i parametri dell'esecuzione:

- `executionContext`: il thread pool utilizzato per l'esecuzione parallela
- `workerCount`: il numero di worker paralleli
- `chunksPerWorker`: il fattore moltiplicativo per la divisione in chunk

In assenza di configurazioni esplicite, la libreria fornisce istanze given predefinite, garantendo il minimo attrito nell'uso per le pipeline standard.

5.1.6 Import semplificato dell'API

Tutti gli elementi dell'API pubblica vengono riesposti direttamente attraverso l'oggetto `FusedStream`. Questo permette di importare l'intera API necessaria con una singola istruzione:

```
1 import FusedStream.*
```

ottenendo così accesso alle funzioni per la costruzione delle pipeline, ai collector terminali pronti all'uso e agli oggetti di configurazione. L'obiettivo è mantenere l'interfaccia compatta e immediata, evitando che l'utente debba importare esplicitamente i diversi oggetti che compongono l'API.

5.2 Design della Stream IR

L'obiettivo della *Stream Intermediate Representation* (Stream IR) è rappresentare la struttura logica di una pipeline e permettere alle successive fasi del compilatore di associare all'albero nuove informazioni. La struttura è prevalentemente lineare, ma può ramificarsi quando un'operazione introduce uno stream annidato, rappresentato mediante un sottoalbero aggiuntivo. Una rappresentazione semplificata della gerarchia è mostrata in Figura 5.3.

Il problema principale da gestire è l'evoluzione dell'Abstract Syntax Tree durante il processo di compilazione: le diverse fasi introducono nuove informazioni da associare ai nodi dell'albero. Una possibile soluzione consiste nel definire una rappresentazione differente per ciascuna fase, duplicando però una parte considerevole della struttura dell'AST. Un'altra opzione consiste nell'utilizzare campi opzionali per le informazioni introdotte successivamente, ma questo richiederebbe controlli sulla loro presenza anche nelle fasi in cui dovrebbero essere sempre disponibili.

Il design scelto per la Stream IR è stato invece una struttura ad albero parametrizzata con *campi indicizzati per fase*, ispirata al paper *Trees That Grow* [5].

```
sealed trait Phase
object Phase {
  final class Raw extends Phase
  final class Enriched extends Phase
}
enum StreamTree[P <: Phase, A] {
  ...
}
```

A Rappresenta il tipo degli elementi prodotti dal nodo.

P Identifica la fase del ciclo di vita (`Phase`) nella quale il nodo si trova.

L'utilizzo della fase come parametro di tipo rende esplicito nel type system lo stato della rappresentazione e, di conseguenza, quali informazioni siano disponibili e quali operazioni siano valide in ciascuna fase. Un *match type* determina il tipo del campo `fields` che fa da container in funzione di `P`. Ogni operatore è quindi rappresentato da un unico caso dell'enumerazione, valido in entrambe le fasi, sul quale è possibile effettuare pattern matching esaustivo.

Tutti gli operatori sono parametrizzati tramite P. Per quelli che non richiedono campi aggiuntivi, come `Map`, il parametro di fase è sufficiente:

```
case Map[P <: Phase, A, B](upstream: StreamTree[P, A]) extends StreamTree[P, B]
```

La fase del nodo impone anche quella del relativo *upstream*, garantendo che l'intera struttura appartenga alla stessa fase. In `FlatMap` lo stesso vincolo si applica anche al sottoalbero `innerStream`.

I nodi che richiedono dati dipendenti dalla fase li esprimono tramite `phase indexed field`. Ad esempio, una sorgente `JListSource` contiene il riferimento alla propria dimensione dopo l'enrichment:

```
case JListSource[P <: Phase, A](
  term: Expr[java.util.List[A]],
```

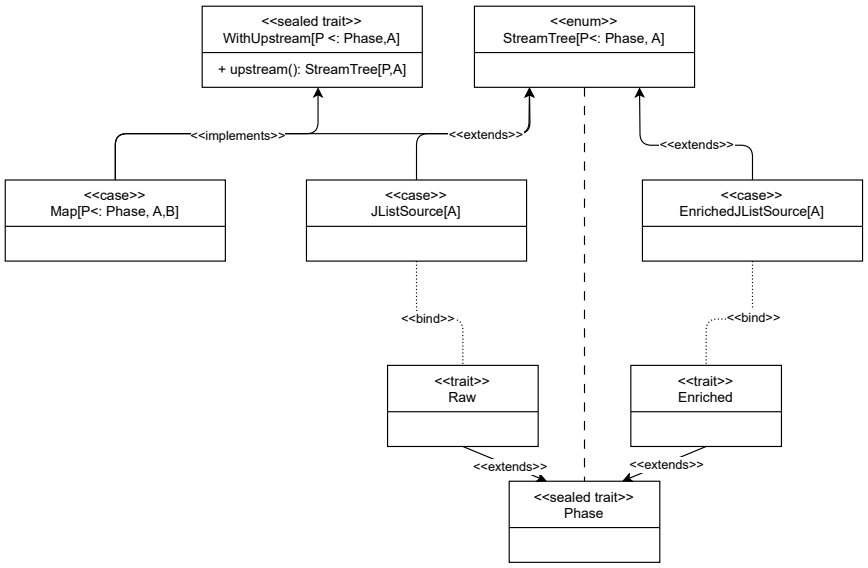


Figura 5.3: Struttura semplificata della Stream IR. I nodi condividono il parametro di fase `P`; la relazione non standard `«bind»` indica la sua specializzazione con `Enriched`, che seleziona il record dei campi aggiuntivi tramite un match type.

```
  outType: Type[A],
  fields: JListFields[P]
) extends StreamTree[P, A]

type JListFields[P <: Phase] = P match {
  case Phase.Raw      => Unit
  case Phase.Enriched => EnrichedJListFields
}
```

Lo stesso schema è applicato agli altri nodi che richiedono campi diversi in funzione della fase.

5.2.1 Ast: phased indexed container

`StreamTree` descrive la pipeline, ma le fasi del compilatore devono scambiarsi altre informazioni, come la strategia terminale e le proprietà statiche della pipeline. Queste informazioni vengono raccolte nello stesso *phased indexed container* `Ast[P, A, R]`, indicizzato per fase come i nodi dell'albero:

```
case class Ast[P <: Phase, A, R](
  stream: StreamTree[P, A],
  collectionStrategy: CollectionStrategy[A, R],
  prefixStatements: List[Statement],
  executionMode: ExecutionMode,
  fields: AstFields[P]
)

type AstFields[P <: Phase] = P match {
  case Phase.Raw      => Unit
  case Phase.Enriched => EnrichedAstFields
}
```

I campi comuni alle due fasi sono:

`stream` L'albero della pipeline nella fase `P`.

`collectionStrategy` La rappresentazione dell'operazione terminale, descritta nella Sezione 5.4.

`prefixStatements` Le istruzioni Scala estratte dai blocchi che racchiudono l'espressione della pipeline.

`executionMode` La modalità `Sequential` o `Parallel`, determinata dal punto di ingresso della macro.

Il parser restituisce un `Ast[Phase.Raw, A, R]` con `fields = ()`. L'enrichment trasforma l'albero e popola i campi aggiuntivi:

`declarations` Le dichiarazioni generate per materializzare espressioni e inizializzare lo stato della pipeline.

`hasAlignedIndexes` Indica se la posizione di un elemento nella sorgente può essere usata direttamente come posizione nel risultato.

`cardinality` Descrive il numero di elementi prodotti come dimensione esatta, limite superiore o quantità sconosciuta. Quando disponibile, la dimensione è rappresentata da un'`Expr[Int]`.

`earlyExitVar` L'eventuale riferimento booleano che il collector modifica quando può arrestare anticipatamente l'elaborazione. `None` indica che il collector non richiede questo stato.

`ref` La lista dei predicati che controllano la prosecuzione dell'elaborazione, comprendente sia l'eventuale condizione del collector sia quelle introdotte dagli operatori della pipeline, come `limit`.

5.3 Rappresentazione per la Code Generation (OP Ir)

La Stream IR descritta nella sezione precedente mantiene una rappresentazione di alto livello della pipeline e dei suoi operatori. Tale struttura è particolarmente adatta alle analisi specifiche del dominio degli stream, ma non costituisce una rappresentazione conveniente per la generazione del codice.

Per questo si introduce una seconda rappresentazione intermedia, *Operation IR* (OP IR). La traduzione verso questa rappresentazione, affidata al `Planner`, costituisce un primo processo di *lowering*: operatori di alto livello vengono eliminati e sostituiti da costrutti che descrivono esplicitamente il modello di esecuzione della pipeline. A differenza della Stream IR, la OP IR non modella quindi il dominio degli stream.

La rappresentazione è organizzata attorno a due categorie fondamentali: *Values*, che descrivono espressioni che producono un valore, e *Operations*, che descrivono azioni e strutture di controllo. Un programma nella OP IR è una sequenza di operazioni e un valore che ne costituisce il risultato finale.

Questo secondo linguaggio intermedio permette di suddividere la generazione del codice in due fasi con responsabilità distinte. Il `Planner` opera sulla rappresentazione logica della pipeline e determina la strategia di esecuzione più adatta esprimendola attraverso la OP IR. Il `Lowerer` opera invece sulle primitive della OP IR e ne effettua il lowering nei corrispondenti costrutti Scala. Le decisioni relative a come implementare la pipeline sono quindi prese prima della generazione effettiva del codice.

5.3.1 Valori

I *Values* rappresentano espressioni che producono un risultato e sono utilizzati come operandi dalle operazioni della OP IR. La gerarchia è composta da valori elementari, come costanti o riferimenti a simboli e da valori composti che descrivono operazioni su altri valori come confronti, operazioni aritmetiche o applicazioni di funzioni.

Un `Value` descrive quindi un'espressione che può essere incorporata all'interno di una `Op`. Ad esempio, la condizione di un ciclo `While` è rappresentata come un `Value[Boolean]`, mentre il valore assegnato da una `AssignVal` è a sua volta rappresentato tramite un `Value`. Un esempio parziale della gerarchia è riportato in Figura 5.4.

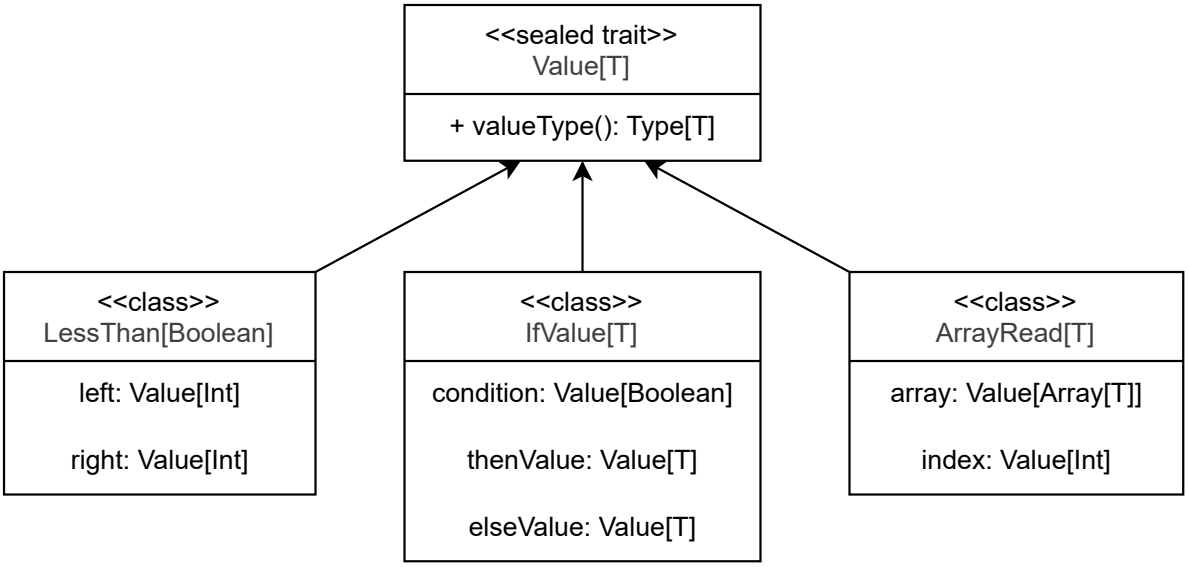


Figura 5.4: Gerarchia e rappresentazione dei valori nella OP IR.

Ogni valore possiede l'informazione relativa al tipo che rappresenta, permettendo quindi di esprimere staticamente i vincoli a livello di tipi tra le diverse primitive della IR. Per esempio in `LessThan`, entrambi gli operandi devono essere valori interi, mentre il risultato è necessariamente un `Value[Boolean]`.

La sola informazione contenuta nel parametro generico non è però sufficiente per la successiva generazione del codice: ogni `Value[T]` espone quindi anche un `Type[T]`, attraverso il campo `valueType`.

Come approfondito nella Sezione 6.4.1, questa informazione viene poi sfruttata durante il lowering per costruire espressioni Scala correttamente tipizzate.

5.3.2 Operazioni

Le *Operations* rappresentano invece le azioni che costituiscono il corpo del programma generato.

La OP IR comprende operazioni elementari per la dichiarazione e l'aggiornamento dello stato, come `Declare[T]`, `AssignVal[T]` o `Inc`. Il controllo del flusso è espresso esplicitamente attraverso `If` e `While`. Nel contesto parallelo, il nodo dedicato `Op.Parallel[OUT]` modella la partizione concorrente del carico e la riduzione finale tramite `ParallelCombine[OUT]`, svincolando la firma del nodo dal tipo interno del buffer di accumulo.

Sono inoltre presenti operazioni specializzate come ad esempio `ArrayWrite` che rappresenta la scrittura di un elemento in un array o `DynamicArrayBuilderAdd` che rappresenta l'inserimento in un buffer.

5.4 La collection strategy

La fase terminale di una stream è rappresentata internamente attraverso una `CollectionStrategy[A, R]` (Figura 5.5). Questa astrazione descrive la strategia con cui gli elementi prodotti dalla pipeline (di tipo `A`) devono essere consumati e aggregati per ottenere il risultato finale di tipo `R`. La firma del tratto è volutamente limitata a due parametri di tipo `[A, R]`, incapsulando il tipo intermedio del buffer `Buf` esclusivamente all'interno della variante generica `WithCollector[A, Buf, R]`.

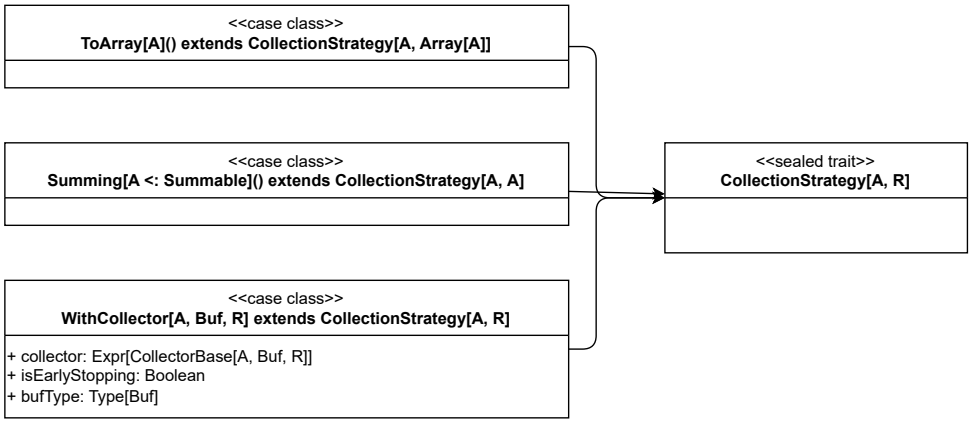


Figura 5.5: Gerarchia `CollectionStrategy[A, R]` e collocazione della strategia in `Ast`. La relazione `«bind»` specializza `P` con `Enriched`, selezionando gli `EnrichedAstFields` che contengono anche le condizioni di terminazione. `Summable` è l'unione dei tipi `Int`, `Long`, `Float` e `Double`.

Capitolo 6

Implementazione

6.1 Parsing

La fase di parsing ha l'obiettivo di trasformare l'albero sintattico prodotto dal compilatore Scala in una `StreamIR` (Sezione 5.2) e di estrarre la `CollectionStrategy` dal `Collector`. Il parsing della pipeline e quello dell'operazione terminale sono mantenuti separati e vengono successivamente combinati nel *phased indexed container* `Ast[Phase.Raw, A, R]` (Sezione 5.2.1), insieme alle istruzioni preliminari e alla modalità di esecuzione. Il campo `fields` è di tipo `Unit` e viene valorizzato con `()`.

6.1.1 Parsing della stream

Il parser riconosce le operazioni all'interno dei nodi dell'AST Scala e ricostruisce ricorsivamente la struttura logica della pipeline. Un esempio completo della trasformazione dall'AST Scala alla corrispondente Stream IR è riportato nella Sezione 6.5.

Rappresentazione del risultato del parsing

Il risultato intermedio è rappresentato da `ParsedTree`, che consente al parser di restituire un albero senza che il chiamante debba conoscerne a priori il tipo di output. Quest'ultimo viene infatti determinato durante il parsing ed è rappresentato mediante l'abstract type member `Out`.

L'implementazione mantiene contemporaneamente `current: StreamTree[Phase.Raw, Out]` e `outType: Type[Out]`. Il primo conserva la relazione statica tra l'albero e il tipo degli elementi prodotti, mentre il secondo ne mantiene l'evidenza richiesta dalle API di `scala.quoted`.

Il tipo può essere recuperato nelle successive operazioni attraverso un path-dependent type:

```
type In = upstream.Out
given Type[In] = upstream.outType
```

Il parser può quindi tipare correttamente funzioni come `In => Boolean` per `filter` o `In => Out` per `map`.

Riconoscimento delle operazioni

Le operazioni della DSL vengono riconosciute mediante pattern matching sulla struttura dell'AST. In forma semplificata:

Listing 6.1: Riconoscimento delle operazioni della DSL

```
expr match
  case Inlined(_, _, inner) => parseTerm(inner)
  case Apply(TypeApply(NamedMethod("from"), _), List(source)) => ...
```

```
  case Apply(Select(upstream, "filter"), List(predicate)) => ...
  case Apply(Select(upstream, "skip"), List(n)) => ...
  case Apply(Select(upstream, "limit"), List(n)) => ...
  case Apply(TypeApply(Select(upstream, "map"), _), List(mapper)) => ...
  case Apply(TypeApply(Select(upstream, "flatMap"), _), List(mapper)) => ...
  case Apply(Select(upstream, "parallel"), _) => parseTerm(upstream)
  case Block(statements, expr) => ...
```

Ogni caso analizza ricorsivamente l'upstream e costruisce il corrispondente nodo della Stream IR. Il tipo di output dell'upstream viene recuperato attraverso `upstream.Out` e la relativa evidenza `upstream.outType`, mantenendo tipizzate sia le funzioni ricevute dalla pipeline sia i nodi della IR.

Parsing ricorsivo di `flatMap`

`flatMap` richiede un trattamento specifico poiché la funzione ricevuta produce a sua volta una `Stream`, che deve essere sottoposta ricorsivamente allo stesso processo di parsing. Si consideri:

```
FusedStream
  .from(numbers)
  .flatMap(x =>
    FusedStream.from(elements(x)).filter(y => y > 0)
  )
```

Il parametro `x` rappresenta un elemento che sarà disponibile soltanto a runtime. Per poter analizzare staticamente la stream interna viene quindi creato un binder tipizzato che ne rappresenta simbolicamente il valore:

```
type In = upstream.Out
given Type[In] = upstream.outType

val flatMapBinder = createConstant[In]("flatMapElem")
val elemRef = Ref(flatMapBinder).asExprOf[In]
```

La funzione viene interpretata come `In => Stream[Out]` e applicata al binder mediante β-reduction:

```
val innerStreamExpr = Expr.betaReduce('{ $function($elemRef) })

val innerTree = parseTerm(innerStreamExpr.asTerm)
```

Si ottengono così due sottoalberi: l'upstream esterno e la stream interna prodotta da `flatMap`. Il nodo della IR conserva inoltre il simbolo del binder che li collega. Durante la successiva code generation il binder verrà associato all'elemento corrente prodotto dall'upstream.

6.1.2 Parsing del collector

L'obiettivo di questa operazione è ricondursi a una collection strategy (approfondita nella Sezione 5.4) a partire dalla rappresentazione ricevuta. Il primo passo consiste nell'analizzarne il tipo associato all'espressione confrontandolo con i collector opachi della libreria. In tal caso il parser costruisce direttamente la strategia corrispondente. Nel caso generale, invece, il collector fornito dall'utente deve essere preservato, viene quindi estratta la *stop policy* per determinare staticamente se il collector supporti lo short-circuiting. Il risultato di questa analisi viene infine incorporato nella strategia generica `WithCollector`. La trasformazione può essere rappresentata come:

$$\texttt{ToArrayCollector[A]} \longrightarrow \texttt{ToArray[A]}$$

$$\texttt{SummingCollector[A]} \longrightarrow \texttt{Summing[A]}$$

$$\texttt{CollectorBase[A, Buf, R]} \longrightarrow \texttt{WithCollector[A, Buf, R]}$$

Nota: il typing su `TerminationPolicy` a questo punto viene rimosso. E' infatti utilizzato solo per definire la grammatica della DSL e viene convertita nel valore booleano `isEarlyStopping` dentro `WithCollector`.

Prima di costruire la strategia generica, se `strictInlining` è abilitato, il parser esegue due controlli separati rifiutando classi anonime e verifica che le implementazioni di `supplier`, `accumulator` e `finisher` siano marcate con `inline`.

6.1.3 Parsing della configurazione statica

La libreria effettua un'ulteriore operazione di parsing relativa alla configurazione di compilazione. A differenza del parsing della stream, basato sull'analisi del TASTy, in questo caso l'estrazione viene effettuata mediante quoted pattern matching. Poiché la configurazione deve essere interamente nota a compile time, i valori dei suoi campi devono poter essere estratti staticamente dall'espressione. Sono quindi accettate istanziazioni di `CompileConfig` i cui argomenti siano costanti note in fase di compilazione, come mostrato dal seguente `FromExpr`:

Listing 6.2: Unapplier con quoted pattern matching per configurazione statica

```
given FromExpr[CompileConfig] with {def unapply(expr: Expr[CompileConfig])(
      using Quotes): Option[CompileConfig] = {
    expr match {
      // CompileConfig(unsafe, logging, strictInlining)
      case '{ CompileConfig($u, $l, $i) } =>
        for {
          unsafe <- u.value
          logging <- l.value
          strictInlining <- i.value
        } yield CompileConfig(unsafe, logging, strictInlining)

      // new CompileConfig(unsafe, logging, strictInlining)
      case '{ new CompileConfig($u, $l, $i) } =>
        for {
          unsafe <- u.value
          logging <- l.value
          strictInlining <- i.value
        } yield CompileConfig(unsafe, logging, strictInlining)
      case _ => None
    }
  }
}
```

6.2 Enrichment

Il *phased indexed container* `Ast[Phase.Raw, A, R]` prodotto dal parser descrive la struttura logica della pipeline, ma non contiene ancora tutte le informazioni necessarie alla generazione del codice. La fase di *enrichment* completa quindi la `StreamTree` con informazioni determinate staticamente e restituisce un `Ast[Phase.Enriched, A, R]` (Sezione 5.2.1). La strategia terminale rimane nel campo comune `collectionStrategy`, mentre le nuove informazioni sono inserite nei campi `fields` dei nodi e negli `EnrichedAstFields` del *phased indexed container*.

L'enrichment svolge tre attività principali: genera le istruzioni di materializzazione delle espressioni che devono essere valutate una sola volta, introduce e propaga lo stato e le condizioni necessarie al controllo dell'esecuzione e determina proprietà globali della pipeline, quali la cardinalità del risultato e l'allineamento degli indici.

6.2.1 Materializzazione delle espressioni

Consiste nell'estrarre le espressioni la cui valutazione deve avvenire una sola volta prima dell'esecuzione della pipeline. Per esempio i parametri delle operazioni `skip` e `limit` quando presenti, vengono materializzati in variabili immutabili. In questo modo un'espressione come:

```
stream.limit(computeLimit())
```

non provoca la rivalutazione di `computeLimit()` a ogni iterazione.

Lo stesso principio viene applicato alle funzioni utilizzate da operatori come `map` e `filter`. Per le sorgenti indicizzabili vengono inoltre materializzate la sorgente e la sua dimensione; il riferimento a quest'ultima viene conservato in `fields.sizeRef` di `JListSource` o `ArraySource`.

6.2.2 Stato e condizioni di terminazione

Durante l'enrichment vengono introdotte le informazioni di stato necessarie all'esecuzione degli operatori. Le operazioni `skip` e `limit` richiedono ad esempio un contatore che rappresenti la posizione corrente nel punto della pipeline in cui sono applicate; il relativo simbolo è conservato in `Slice.fields.counterRefSymbol`. Nel caso di `limit`, lo stato ha anche effetto sulla terminazione della pipeline: raggiunto il numero massimo di elementi non è più necessario interrogare la sorgente. La condizione:

$$\texttt{sliceCounter} < \texttt{until}$$

viene quindi aggiunta all'insieme dei predicati di terminazione e propagata verso i nodi a monte.

Lo stesso meccanismo viene utilizzato per i collector *short-circuiting*. Quando la strategia di raccolta consente la terminazione anticipata, viene introdotta una variabile mutabile che rappresenta la possibilità di continuare l'elaborazione. Il riferimento è conservato in `Ast.fields.earlyExitVar`, mentre la lista dei predicati propagati alla sorgente della pipeline è raccolta in `Ast.fields.ref`. Quando il collector determina che il risultato è completo, tale variabile viene modificata, causando l'arresto dei cicli interessati. La condizione di un ciclo generato non è quindi costituita solamente dalla condizione della sorgente, ma dalla combinazione con tutti i predicati propagati:

$$C_{\text{loop}} = C_{\text{source}} \wedge p_1 \wedge p_2 \wedge \cdots \wedge p_n.$$

`FlatMap` rappresenta un caso particolare: il relativo nodo contiene sia l'upstream della pipeline esterna sia la stream interna prodotta dalla funzione. Entrambi gli alberi vengono trasformati nella fase `Enriched`; le dichiarazioni e i predicati ottenuti dall'analisi della stream interna vengono conservati rispettivamente in `FlatMap.fields.innerMaterialized` e `FlatMap.fields.predicates`.

L'enrichment determina pertanto il livello di annidamento al quale stato e condizioni dovranno essere collocati durante la successiva generazione del codice: lo stato appartenente alla stream interna deve essere infatti inizializzato nuovamente per ciascun elemento prodotto dalla stream esterna. Ad esempio, il contatore associato a un `limit` interno alla `flatMap` non può essere dichiarato globalmente, ma deve essere ricreato all'interno della corrispondente iterazione. Viceversa, una condizione di terminazione introdotta a valle della `flatMap` deve poter arrestare sia l'iterazione interna sia quella esterna.

6.2.3 Analisi della cardinalità

Terminato l'enrichment dell'albero vengono determinate due proprietà utilizzate per selezionare la strategia di generazione: la cardinalità del risultato e l'allineamento degli indici. La cardinalità è rappresentata mediante:

```
enum Cardinality {
  case Exact(size: Expr[Int])
  case UpperBound(size: Expr[Int])
  case Unknown
}
```

Una sorgente indicizzabile, come un array o una lista, possiede cardinalità esatta. Operazioni come `map` la preservano, mentre `filter`, potendo eliminare elementi ma non aumentarne il numero, la trasformano in un limite superiore. Sorgenti genericamente iterabili e operazioni come `flatMap`, per le quali non è possibile determinare staticamente il numero di elementi prodotti, conducono invece a una cardinalità sconosciuta.

A questa informazione si affianca `hasAlignedIndexes`, ovvero se, in caso di sorgenti indicizzate, l'indice dell'elemento nella sorgente corrisponde all'indice nel risultato. La proprietà è inizialmente valida per sorgenti indicizzabili ed è preservata da operazioni come `map` e `limit`; viene invece invalidata da `filter`, `skip` e `flatMap`, che rompono la corrispondenza tra posizione letta e posizione prodotta. Entrambe le proprietà vengono conservate negli `EnrichedAstFields`, rispettivamente come `cardinality` e `hasAlignedIndexes`.

6.3 Planning

Una volta completato l'enrichment, la Stream IR contiene le informazioni necessarie per determinare come la pipeline debba essere eseguita. Il `Planner` accetta soltanto la fase `Enriched` del *phased indexed container* `Ast`: legge la strategia terminale da `collectionStrategy` e le proprietà della pipeline da `fields`. Il suo metodo `execute` traduce questa rappresentazione logica nella *Operation IR* (OP IR), introdotta nella Sezione 5.3.

È in questa fase che vengono prese le principali decisioni relative al piano di esecuzione. Il piano dipende infatti dall'operazione terminale, dalla modalità sequenziale o parallela, dalla cardinalità della pipeline, dall'allineamento degli indici e dalle caratteristiche della sorgente. Il risultato è un `Program` composto da operazioni e da un valore finale, nel quale i concetti propri del dominio degli stream sono stati tradotti in dichiarazioni, condizioni, cicli e operazioni sui valori. Il `Lowerer`, descritto nella Sezione 6.4, può quindi limitarsi alla traduzione meccanica di tali operazioni verso l'AST Scala.

6.3.1 Selezione della strategia terminale

Il primo livello decisionale riguarda l'operazione terminale. La `CollectionStrategy` ottenuta durante il parsing distingue tre casi:

- `ToArray`, per il quale viene selezionata una strategia di materializzazione dipendente dalla cardinalità della pipeline;
- `Summing`, tradotto in un accumulatore numerico specializzato sul tipo degli elementi;
- `WithCollector`, che utilizza il protocollo generale `supplier`/`accumulator`/`finisher`.

Nel caso del terminale `toArray`, per esempio, la strategia dipende principalmente dalla cardinalità e dall'allineamento degli indici. Con cardinalità esatta e indici allineati, è possibile preallocare l'array finale e utilizzare direttamente l'indice della sorgente. La stessa proprietà è particolarmente utile in parallelo: poiché la posizione finale di ogni elemento è nota

indipendentemente, i worker possono scrivere in regioni distinte dello stesso array mediante `ParallelCombine.ArrayDirect()`, senza produrre risultati intermedi. Quando è noto solamente un limite superiore, ad esempio in presenza di `filter`, viene comunque preallocato un array, ma viene mantenuto un indice di output separato. Al termine l'array viene eventualmente troncato al numero di elementi effettivamente prodotti. Se invece la cardinalità è sconosciuta, come per una sorgente genericamente iterabile o per `flatMap`, viene utilizzato un buffer dinamico e l'array viene materializzato solamente al termine dell'elaborazione.

6.3.2 Generazione della pipeline e delle sorgenti

Determinata la strategia terminale, viene generato il corpo della pipeline. La generazione segue un modello basato su una funzione di emissione:

```
private type Emit[A] = Emitted[A] => Op

private final case class Emitted[A](
  elem: Value[A],
  sourceIndex: Option[Value[Int]]
)
```

La funzione `Emit` rappresenta il comportamento del downstream quando il nodo corrente produce un elemento. Gli operatori intermedi non generano quindi un ciclo autonomo, ma trasformano questa continuation e delegano la generazione al proprio upstream. Il ciclo viene emesso solamente quando la ricorsione raggiunge la sorgente.

Ad esempio, `filter` condiziona l'invocazione del downstream:

$$E'(x) = \texttt{if } p(x) \texttt{ then } E(x),$$

che nella OP IR corrisponde a:

```
If(
  ApplyFun(filter.predicate, emitted.elem),
  emit(emitted)
)
```

Una `map` modifica invece il valore propagato:

```
Declare(mapped, ApplyFun(map.function, emitted.elem))
emit(Emitted(SymbolRef(mapped), emitted.sourceIndex))
```

Il caso più articolato è `flatMap`. Il nodo contiene infatti una seconda Stream IR che rappresenta la pipeline interna. Il binder creato durante il parsing viene associato all'elemento corrente dell'upstream:

```
Declare(flatMap.elemSymbol, emitted.elem)
```

La stream interna viene quindi tradotta ricorsivamente utilizzando la stessa continuation del `flatMap`. Il risultato è l'annidamento diretto del ciclo interno nel ciclo esterno, senza materializzare una stream intermedia. Concettualmente:

```
while (outerCondition) {
  val binder = nextOuter()
  while (innerCondition) {
    val y = nextInner(binder)
    if (p(y))
```

```
        emit(y)
    }
}
```

La ricorsione termina infine quando viene raggiunta la sorgente, che introduce il ciclo effettivo. La tipologia concreta della sorgente può determinare ulteriori specializzazioni. Per una `java.util.List`, ad esempio, viene distinto il caso `ArrayList`, per il quale l'accesso indicizzato evita la creazione dell'iteratore e, se abilitato dalla configurazione di compilazione (`useUnsafe`), è inoltre possibile accedere direttamente all'array interno tramite `VarHandle`.

6.3.3 Generazione del piano parallelo

Nel caso parallelo, la OP IR raccoglie esplicitamente le informazioni necessarie a descrivere la partizione del lavoro:

- l'intervallo della sorgente interessato dalla computazione;
- la sequenza di operazioni da eseguire su ciascuna partizione;
- l'eventuale risultato locale prodotto da ogni partizione;
- la strategia di combinazione dei risultati parziali;
- il riferimento al risultato finale del calcolo.

La OP IR non descrive quindi i dettagli del meccanismo di scheduling: rappresenta il parallelismo come operazione di alto livello, lasciando al `Lowerer` la sua traduzione nelle primitive runtime.

6.4 Lowering

La classe `Lowerer` realizza la fase di lowering, l'ultimo passaggio del compilatore: il metodo `lower` riceve un `Program[T]` e produce un'espressione Scala `Expr[T]`, che viene restituita al compilatore Scala. A questo punto le decisioni relative alla strategia di esecuzione sono già state prese dal `Planner`: il `Lowerer` traduce l'OP IR nei corrispondenti nodi dell'AST Scala. La trasformazione è implementata principalmente attraverso due funzioni: `lowerOp`, per le operazioni, e `lowerValue`, per le espressioni che producono un valore.

6.4.1 Traduzione dell'OP IR

La traduzione viene effettuata mediante pattern matching esaustivo sui nodi dell'OP IR, le operazioni vengono ricondotte ai corrispondenti costrutti dell'AST Scala. Analogamente, i nodi `Value[T]` vengono tradotti ricorsivamente in `Expr[T]`. Riferimenti, costanti ed espressioni Scala già disponibili possono essere convertiti direttamente, mentre gli operatori composti richiedono il lowering ricorsivo dei propri operandi. Ad esempio:

```
LessThan(left, right)
```

viene tradotto in:

```
'{
  ${lowerValue(left)} < ${lowerValue(right)}
}
```

Lo stesso procedimento viene applicato alle operazioni logiche, confronti, accessi ad array e alle operazioni sui collector, eliminando progressivamente l'OP IR fino a ottenere esclusivamente nodi appartenenti all'AST Scala.

Questa trasformazione conserva le informazioni di tipo: per costruire un'`Expr[T]`, Scala richiede che sia disponibile un'evidenza `Type[T]`. Per questo motivo, come descritto nella Sezione 5.3, ciascun `Value[T]` conserva, oltre al parametro di tipo, anche il `Type[T]` corrispondente nel campo `valueType`. Durante il pattern matching sull'OP IR tale informazione viene recuperata e resa disponibile localmente. Ad esempio, nel lowering di una scrittura su array:

```
case write: ArrayWrite[t] =>
  given Type[t] = write.valueType

  val array = lowerValue[Array[t]](write.array)
  val index = lowerValue[Int](write.index)
  val value = lowerValue[t](write.value)

  '{ $array($index) = $value }
```

L'informazione conservata nella rappresentazione intermedia permette quindi di mantenere il collegamento tra il sistema di tipi dell'OP IR e quello delle Scala Quotes fino all'ultima fase della generazione.

6.4.2 Applicazione e inlining delle funzioni

Un caso particolarmente importante è costituito da `ApplyFun`. In questo caso viene utilizzata la β-reduction, descritta nella Sezione 3.3.3. Quando la funzione è una lambda, la β-reduction ne inserisce il corpo direttamente nel punto di utilizzo. Ad esempio:

```
ApplyFun(x => x * 2, elem)
```

viene ridotto a:

```
elem * 2
```

In questo modo la funzione non deve essere mantenuta come entità invocabile nel codice generato, eliminando il relativo dispatch.

6.4.3 Chiamate e inlining dei collector

Le operazioni sui collector vengono tradotte generando l'istruzione dell'AST che chiama il metodo associato sull'oggetto collector, ad esempio per l'accumulazione:

```
Select.overloaded(collector.asTerm, "accumulator", Nil, arguments).asExprOf[T]
```

Questo permette al compilatore Scala di fare inlining dei metodi (quando dichiarati `inline def`). A differenza della β-reduction utilizzata per `ApplyFun` in questo caso si lascia l'inlining al compilatore Scala. I factory methods dei collector predefiniti sono `transparent inline`, così da esporre il tipo delle istanze create.

6.4.4 Lowering di pipelines parallele

Il nodo `Parallel` costituisce il caso più articolato del lowering. Il `Planner` ha già determinato la strategia di esecuzione e la modalità di combinazione dei risultati; il `Lowerer` deve quindi

trasformare tale piano nel corrispondente codice concorrente. Il tipo di `ParallelCombine` determina l'implementazione specializzata da generare. La libreria prevede tre strategie specializzate e una generica:

- `Sum`: combina i risultati parziali mediante somma;
- `ArrayConcat`: concatena gli array prodotti dai singoli chunk;
- `ArrayDirect`: scrive direttamente nell'array finale condiviso;
- `GenericCombiner`: combina i risultati tramite il `Combinable` associato a un `Collector` generico.

L'intervallo della sorgente viene partizionato in chunk. Quando la strategia richiede risultati intermedi, questi vengono memorizzati separatamente per ciascun chunk e successivamente combinati. Nel caso della somma, ad esempio:

```
partials(chunkIdx) = localResult
```

Al termine dell'elaborazione i risultati parziali vengono ridotti sequenzialmente per ottenere il risultato finale.

Anche lo scheduling viene definito a runtime sulla base della dimensione della sorgente e della configurazione. Il numero di chunk è limitato dalla dimensione dell'input e determinato in funzione del numero di worker. L'assegnazione non è statica: ogni worker acquisisce un chunk e, terminata l'elaborazione, richiede dinamicamente il successivo attraverso un contatore atomico condiviso. In questo modo i chunk possono essere distribuiti progressivamente tra i worker senza fissarne preventivamente l'assegnazione e bilanciando dinamicamente il carico. Non è presente un meccanismo di work-stealing.

6.5 Esempio di traduzione end-to-end

Questa sezione ripercorre passo passo il processo di analisi e generazione del codice effettuato dalla libreria a partire da una semplice pipeline. L'intero processo segue il flusso dell'applicativo:

Scala AST ⟶ StreamIR ⟶ Enriched StreamIR ⟶ OpIR ⟶ Scala AST ⟶ JVM bytecode.

6.5.1 L'input alla libreria

La pipeline utilizzata come esempio è mostrata nel Listato 6.5.1:

```
FusedStream.from(numbers)
  .map((i: Int) => i * 2)
  .filter((i: Int) => i % 2 == 0)
  .toArray
```

La macro riceve l'espressione sotto forma di AST Scala tipizzato; in particolare, il parser riceve l'albero mostrato nel Listato 6.5.1:

```
Inlined(
  None,
  Nil,
  Apply(
    Select(
      Apply(
```

```
        Apply(
          TypeApply(
            Select(
              Apply(
                TypeApply(
                  Select(Ident("FusedStream"), "from"),
                  List(Inferred())
                ),
                List(Ident("numbers"))
              ),
              "map"
            ),
            List(Inferred())
          ),
          List(
            Block(
              List(
                DefDef(
                  "$anonfun",
                  List(TermParamClause(List(ValDef("i", Inferred(), None)))),
                  Inferred(),
                  Some(
                    Apply(
                      Select(Ident("i"), "*"),
                      List(Literal(IntConstant(2)))
                    )
                  )
                )
              ),
              Closure(Ident("$anonfun"), None)
            )
          )
        ),
        List(
          Apply(
            TypeApply(Ident("given_NotMapToStream_T"), List(Inferred())),
            List(Ident("value"))
          )
        )
      ),
      "filter"
    ),
    List(
      Block(
        List(
          DefDef(
            "$anonfun",
            List(TermParamClause(List(ValDef("i", Inferred(), None)))),
            Inferred(),
            Some(
              Apply(
                Select(
                  Apply(
                    Select(Ident("i"), "%"),
                    List(Literal(IntConstant(2)))
                  ),
                  "=="
                ),
```

```
                List(Literal(IntConstant(0)))
              )
            )
          )
        ),
        Closure(Ident("$anonfun"), None)
      )
    )
  )
)
```

E l'espressione rappresentante il terminale:

```
fuse.FusedStream.toArray[scala.Int]
```

6.5.2 Il risultato del parsing

La prima fase consiste nella traduzione dell'AST Scala nella rappresentazione intermedia ad alto livello descritta nella Sezione 5.2. Il *phased indexed container* `Ast[Phase.Raw, Int, Array[Int]]` corrispondente all'AST del Listato 6.5.1 è riportato schematicamente nel Listato 6.3. Le espressioni Scala incorporate nella IR sono indicate tramite la notazione delle Quotes:

Listing 6.3: Ast in fase Raw ottenuto al termine del parsing

```
Ast[Phase.Raw, Int, Array[Int]](
  stream = Filter[Phase.Raw, Int](
    upstream = Map[Phase.Raw, Int, Int](
      upstream = JListSource[Phase.Raw, Int](
        term = '{ numbers },
        outType = Type.of[Int],
        fields = ()
      ),
      function = '{ (i: Int) => i * 2 },
      inType = Type.of[Int],
      outType = Type.of[Int]
    ),
    predicate = '{ (i: Int) => i % 2 == 0 },
    outType = Type.of[Int]
  ),
  collectionStrategy = ToArray[Int](),
  prefixStatements = Nil,
  executionMode = ExecutionMode.Sequential,
  fields = ()
)
```

Per questa pipeline vengono determinate la strategia terminale `ToArray[Int]` e la modalità `Sequential`. Non sono presenti istruzioni preliminari. I campi `fields` di `Ast` e di `JListSource` sono di tipo `Unit`; `Map` e `Filter` non richiedono campi aggiuntivi.

6.5.3 L'output dell'enrichment

La `StreamIR` del Listato 6.3 viene quindi sottoposta alla fase di enrichment (Sezione 6.2). Il risultato è un `Ast[Phase.Enriched, Int, Array[Int]]`, illustrato schematicamente nel Listato 6.4. `sourceSymbol` e `sizeSymbol` rappresentano i simboli tipizzati generati per la sorgente e la sua dimensione; `sourceRef` e `sizeRef` sono le corrispondenti `Expr`:

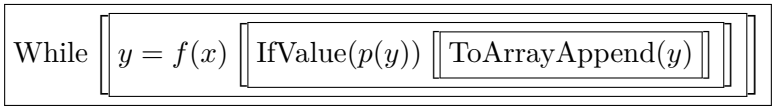
Listing 6.4: Ast in fase Enriched dopo l'enrichment

```
Ast[Phase.Enriched, Int, Array[Int]](
  stream = Filter[Phase.Enriched, Int](
    upstream = Map[Phase.Enriched, Int, Int](
      upstream = JListSource[Phase.Enriched, Int](
        term = sourceRef,
        outType = Type.of[Int],
        fields = EnrichedJListFields(sizeRef)
      ),
      function = '{ (i: Int) => i * 2 },
      inType = Type.of[Int],
      outType = Type.of[Int]
    ),
    predicate = '{ (i: Int) => i % 2 == 0 },
    outType = Type.of[Int]
  ),
  collectionStrategy = ToArray[Int](),
  prefixStatements = Nil,
  executionMode = ExecutionMode.Sequential,
  fields = EnrichedAstFields(
    declarations = List(
      Declaration.Impl(sourceSymbol, '{ numbers }),
      Declaration.Impl(sizeSymbol, '{ $sourceRef.size() })
    ),
    hasAlignedIndexes = false,
    cardinality = Cardinality.UpperBound(sizeRef),
    earlyExitVar = None,
    ref = Nil
  )
)
```

La presenza di `filter` rende la dimensione della sorgente un limite superiore e invalida l'allineamento degli indici. La strategia `ToArray` non richiede arresto anticipato e la pipeline non contiene `limit`, perciò `earlyExitVar` è `None` e `ref` è vuota. Le dichiarazioni sono ancora rappresentate da `Declaration.Impl`: verranno tradotte in operazioni `Declare` durante la generazione della OP IR.

6.5.4 Pianificazione in OP IR

A partire dalla rappresentazione del Listato 6.4, il `Planner` determina la strategia con cui attraversare la sorgente ed eseguire le operazioni della pipeline. Il piano di esecuzione assume la forma:



Il piano di esecuzione viene espresso tramite `OpIR` nel Listato 6.5:

Listing 6.5: OPIR generato per una pipeline con mapping, filtro e raccolta in array

```
statements = [
  // Estrae la lista di origine
  Declare(source$macro$1,ScalaExpr(numbers)),
  // Estrae la size della lista
  Declare(sourceSize$macro$1,ScalaExpr(source$macro$1.size())),
  // Crea un vettore preallocato
```

```
  Declare(vec$macro$1,ArrayDefine[scala.Int](ScalaExpr(sourceSize$macro$1))),
  // Dichiara il counter e inizializza a 0
  Declare(i$macro$1,ConstantVal(0)),
  // Genera codice ottimizzato a runtime: se e' una ArrayList usiamo accesso
    indicizzato (costo O(1), niente iteratori)
  If(
    condition =IsInstanceOf[java.util.ArrayList[]](ScalaExpr(source$macro$1)),
    then =
      CodeBlock(
        Declare(i$macro$2,ConstantVal(0)),
        While(
          // Ciclo diretto su size, non abbiamo condizioni come limit o skip
          condition =LessThan(SymbolRef(i$macro$2),ScalaExpr(sourceSize$macro$1)),
          body =
            CodeBlock(
              // Leggiamo valore della list in una nuova variabile.
              // Se useUnsafe = true allora JListRead legge direttamente
              // l'array sottostante ad arraylist tramite varhandle
              Declare(elem$macro$1,
                      JListRead(ScalaExpr(source$macro$1),
                      SymbolRef(i$macro$2))),
              // Map
              CodeBlock(
                Declare(
                  mapped$macro$1,
                  ApplyFun(((i: scala.Int) => i.*(2)),SymbolRef(elem$macro$1))
                ),
                // Filter
                If(
                  condition =ApplyFun(
                        ((i: scala.Int) => i.%(2).==(0)),
                        SymbolRef(mapped$macro$1)),
                  then =
                    // Se la condizione e' rispettata write all'indice
                    CodeBlock(
                      ArrayWrite(
                        SymbolRef(vec$macro$1),
                        SymbolRef(i$macro$1),
                        SymbolRef(mapped$macro$1)),
                      Inc(i$macro$1)
                    )
                )
              ),
              Inc(i$macro$2)
            )
        )
      ),
    //codice non ottimizzato, per List senza accesso diretto, usa iteratore
    else =...
  )
],
result =  IfValue(
  // In base alla size dell'array lo ritorno direttamente o lo tronco
  Equal(SymbolRef(i$macro$1),ArrayLength(SymbolRef(vec$macro$1))),
  SymbolRef(vec$macro$1),
  ArrayTake(SymbolRef(vec$macro$1),SymbolRef(i$macro$1))
)
```

6.5.5 Lowering verso l'AST Scala

Il `Lowerer` traduce poi il programma della OP IR nell'AST Scala (qui omesso vista la verbosità e complessità). Segue invece una decompilazione del bytecode Java corrispondente al codice generato:

```
ArrayList<Integer> numbers = new ArrayList<Integer>();
numbers.addAll(List.of(..));

int[] nArray;
RuntimeConfig conff = RuntimeConfig$.MODULE$.default();
ArrayList<Integer> source$macro$1 = numbers;
int sourceSize$macro$1 = source$macro$1.size();
int[] vec$macro$1 = new int[sourceSize$macro$1];
int i$macro$1 = 0;
if (source$macro$1 instanceof ArrayList) {
    ArrayList<Integer> list$proxy1 = source$macro$1;
    Object[] raw$macro$1 = ArrayListAccessor$.MODULE$.inline$ElementDataHandle().
  get(list$proxy1);
    for (int i$macro$2 = 0; i$macro$2 < sourceSize$macro$1; ++i$macro$2) {
        int elem$macro$1 = BoxesRunTime.unboxToInt(raw$macro$1[i$macro$2]);
        int mapped$macro$1 = elem$macro$1 * 2;
        if (mapped$macro$1 % 2 != 0) continue;
        vec$macro$1[i$macro$1] = mapped$macro$1;
        ++i$macro$1;
    }
} else {
    // tramite foreach, rimosso per brevita'
}
if (i$macro$1 == vec$macro$1.length) {
    nArray = vec$macro$1;
} else {
    Object object = Predef$.MODULE$.genericArrayOps(vec$macro$1);
    nArray = (int[])ArrayOps$.MODULE$.take$extension(object, i$macro$1);
}
int[] result = nArray;
```

6.5.6 Il bytecode JVM generato

La corretta fusione delle operazioni può essere verificata anche ispezionando direttamente il bytecode JVM generato (riportato nel Listato 6.6):

Listing 6.6: Estratto del bytecode JVM relativo al corpo della pipeline

```
125: iload         10
127: iload         5
129: if_icmpge     175

132: aload         8
134: iload         10
136: aaload
137: invokestatic  BoxesRunTime.unboxToInt
140: istore        11

142: iload         11
144: iconst_2
145: imul
```

```
146: istore        12

148: iload         12
150: iconst_2
151: irem
152: iconst_0
153: if_icmpne     169

156: aload         6
158: iload         7
160: iload         12
162: iastore
163: iinc          7, 1

169: iinc          10, 1
172: goto          125
```

Il risultato mostra in maniera evidente l'effetto della fusione e dell'inlining. L'operazione di mapping

$$x \mapsto x \cdot 2$$

non corrisponde più all'invocazione di una funzione, ma viene tradotta direttamente nell'istruzione JVM `imul`. Analogamente, il predicato

$$x \bmod 2 = 0$$

è rappresentato dalle istruzioni `irem` e `if_icmpne`.

Nel corpo del ciclo non sono presenti invocazioni dinamiche verso le funzioni utilizzate da `map` e `filter`. Non compaiono né chiamate a `Function.apply`, né istruzioni `invokedynamic` associate alla costruzione o all'invocazione di lambda. Anche la raccolta del risultato è ridotta a operazioni primitive: `iastore` scrive direttamente il valore trasformato nell'array di output e `iinc` aggiorna il relativo indice.

Capitolo 7

Testing e valutazione delle prestazioni

7.1 Testing

Il testing deve affrontare due problemi:

1. **Correttezza a runtime**: accertarsi che il codice generato dalle macro produca i risultati attesi.
2. **Verifica a compile-time**: assicurarsi che le combinazioni non valide vengano intercettate subito dal compilatore, fornendo messaggi di errore chiari.

I test sono scritti con `MUnit` e sono organizzati su più livelli separando i controlli interni da quelli sull'API pubblica:

- **Unit test interni**: verificano i singoli stadi del motore di compilazione in isolamento.
- **Test funzionali end-to-end**: compilano ed eseguono pipeline complete scritte tramite l'API pubblica, verificando il risultato dell'esecuzione. Questi test sono suddivisi in quattro suite dedicate:
  - `SequentialStreamTest`: trasformazioni sequenziali di base e operatori di slice;
  - `PipelineSemanticsTest`: ordine di valutazione delle espressioni, isolamento dei side-effect e correttezza della fusione;
  - `SourceAndCollectorTest`: sorgenti Scala e Java, ottimizzazioni con accesso diretto alla memoria e collector terminali;
  - `ParallelStreamTest`: partizionamento del carico su più worker, concorrenza e riduzioni parallele.
- **Test di compilazione**: verificano che l'uso scorretto dell'API venga bloccato a tempo di compilazione.

Copertura

La copertura del codice è stata misurata tramite `sbt-scoverage` sul modulo principale (`core`), come riassunto in Tabella 7.1. La suite raggiunge una copertura complessiva delle istruzioni del **91.31%** (1733 su 1898 istruzioni) e una copertura dei rami decisionali del **91.67%**.

Tabella 7.1: Copertura del codice rilevata da `sbt-scoverage` sul modulo `core`.

Package / Sottosistema	**Istruzioni**	**Coverage**
fuse.internal.parallel	282 / 282	100.00%
fuse.internal	1336 / 1454	91.88%
fuse.internal.ir	86 / 94	91.49%
fuse (API pubblica)	29 / 68	42.65%
Totale (Branch: 91.67%)	**1733 / 1898**	**91.31%**

7.2 Valutazione delle prestazioni

7.2.1 Metodologia

Le misurazioni sono state condotte con Java Microbenchmark Harness (JMH) [6], su OpenJDK 64-Bit (21.0.7-LTS) x86_64 (Windows). I dati riportati provengono da `benchmark results.md`, aggiornato al 6 ottobre 2026. La configurazione prevede:

- heap iniziale e massimo di 8 GB (`-Xms8g -Xmx8g`);
- 4 processori disponibili alla JVM (`-XX:ActiveProcessorCount=4`);
- 3 fork indipendenti, ciascuno con 5 iterazioni di warm-up e 5 di misura, tutte da 2 secondi: 15 campioni per punto sperimentale.

La sorgente è un'`ArrayList` di transazioni con $N \in \{10, 100, 1\,000, 100\,000\}$. Si confrontano `FusedStream`, cicli manuali, Java Stream e Scala View. Per `FusedStream` si distinguono la configurazione predefinita (`useUnsafe = true`) e quella Safe (`useUnsafe = false`).

Le tabelle riportano il tempo medio in microsecondi per operazione e il margine di errore restituito da JMH; valori inferiori indicano prestazioni migliori. Negli scenari megamorfici, `@OperationsPerInvocation(8)` normalizza il tempo delle otto query eseguite per invocazione: il valore è quindi riferito a una singola query, mediato sulle otto. Salvo diversa indicazione, i risultati sono ottenuti a Tier 4.

7.2.2 Benchmark sequenziali

Scenario monomorfico

Il primo scenario esegue una sola pipeline, che estrae e somma gli importi delle transazioni (`map` seguito da `sum`). I call site delle funzioni osservano un unico target, favorendo l'inlining da parte del JIT.

Tabella 7.2: Scenario monomorfico a Tier 4: tempo medio (μs/op).

Implementazione	$N = 10$	$N = 100$	$N = 1\,000$	$N = 100\,000$
FusedStream (Unsafe)	0.006 ± 0.001	0.067 ± 0.001	0.897 ± 0.002	92.760 ± 0.224
FusedStream (Safe)	0.007 ± 0.001	0.067 ± 0.001	0.896 ± 0.001	94.040 ± 1.072
Java Manual Loop	0.007 ± 0.001	0.067 ± 0.001	0.896 ± 0.002	92.869 ± 0.257
Java Stream	0.067 ± 0.001	0.451 ± 0.007	4.279 ± 0.007	428.277 ± 1.573
Scala View	0.041 ± 0.001	0.317 ± 0.005	3.115 ± 0.023	320.285 ± 4.191

La Tabella 7.2 mostra tempi sostanzialmente allineati tra `FusedStream`, la variante Safe e il ciclo manuale. Su 100 000 elementi, `FusedStream` richiede 92.760 μs, contro 92.869 μs del ciclo manuale; Java Stream e Scala View richiedono rispettivamente circa 4.62 e 3.45 volte il tempo della versione fused.

Scenario megamorfico

Il secondo scenario esegue otto query eterogenee di filtro, trasformazione e somma sulle stesse transazioni. Le query sequenziali sono definite in metodi statici separati. La presenza di lambda differenti sollecita i call site condivisi delle librerie a runtime, rendendo meno efficace l'inlining speculativo; `FusedStream` incorpora invece le lambda nel codice generato a compile-time.

Tabella 7.3: Scenario megamorfico sequenziale a Tier 4: tempo medio per query (μs/op).

Implementazione	$N = 10$	$N = 100$	$N = 1\,000$	$N = 100\,000$
FusedStream (Unsafe)	0.011 ± 0.001	0.082 ± 0.001	1.267 ± 0.004	114.744 ± 2.171
FusedStream (Safe)	0.012 ± 0.001	0.083 ± 0.002	1.272 ± 0.010	117.412 ± 1.863
Java Manual Loop	0.011 ± 0.001	0.082 ± 0.001	1.260 ± 0.003	115.942 ± 1.505
Java Stream	0.084 ± 0.008	0.511 ± 0.026	4.688 ± 0.141	474.972 ± 17.787
Scala View	0.062 ± 0.001	0.492 ± 0.011	4.943 ± 0.337	631.320 ± 4.161

Anche in questo scenario `FusedStream` mantiene tempi vicini al ciclo manuale (Tabella 7.3). Con $N = 100\,000$, i tempi sono 114.744 μs per la versione predefinita, 117.412 μs per la variante Safe e 115.942 μs per il ciclo manuale. Rispetto alla versione predefinita, Java Stream richiede circa 4.14 volte il tempo e Scala View circa 5.50 volte. L'accesso Safe produce risultati simili con C2 attivo; la sua incidenza ai livelli JIT inferiori è discussa nella Sezione 7.2.4.

7.2.3 Benchmark paralleli

Lo scenario parallelo esegue le stesse otto query. Si confrontano `FusedStream` con `parallel()`, Java Parallel Stream e una versione manuale che suddivide i dati in chunk e sottopone task a un `ForkJoinPool`. La JVM dispone di 4 processori in tutte le configurazioni.

Tabella 7.4: Scenario megamorfico parallelo a Tier 4: tempo medio per query (μs/op).

Implementazione	$N = 10$	$N = 100$	$N = 1\,000$	$N = 100\,000$
FusedStream Parallel	11.221 ± 0.969	11.893 ± 1.009	14.037 ± 0.455	48.199 ± 0.318
Java Manual Parallel	11.077 ± 0.780	11.976 ± 0.951	13.345 ± 0.505	66.263 ± 0.770
Java Parallel Stream	3.772 ± 0.051	5.124 ± 0.220	8.668 ± 0.516	148.636 ± 3.138

Su 100 000 elementi, `FusedStream` passa da 114.744 a 48.199 μs per query: lo speedup è 2.38×, con un'efficienza del 59.5% rispetto al riferimento ideale di 4×. La versione manuale raggiunge 1.75× e Java Parallel Stream 3.20×. Quest'ultima scala meglio rispetto al proprio sequenziale, ma impiega comunque circa 3.08 volte il tempo di `FusedStream` parallelo.

Per $N \leq 1\,000$, il costo di coordinamento supera il beneficio della suddivisione del lavoro: tutte le versioni parallele risultano più lente delle corrispondenti sequenziali. Java Parallel Stream ha tempi inferiori alle altre versioni parallele su queste taglie. Il vantaggio del parallelismo dipende quindi dalla dimensione e dal costo della computazione.

7.2.4 Livelli di compilazione JIT

Per valutare la dipendenza dalle ottimizzazioni della JVM, i benchmark sono stati ripetuti limitando HotSpot tramite `-XX:TieredStopAtLevel`: Tier 0 esegue il bytecode nell'interprete, Tier 1 usa C1 senza profilazione e Tier 4 consente la compilazione C2. La Tabella 7.5 riassume il confronto sullo scenario megamorfico di taglia maggiore.

A Tier 0, l'accesso diretto al vettore interno dell'`ArrayList` favorisce la configurazione predefinita: 6251.653 μs contro 11451.373 μs della variante Safe e 10971.256 μs del ciclo manuale. La variante Safe usa `get(i)`, come il ciclo manuale, e presenta tempi più vicini a quest'ultimo.

Tabella 7.5: Scenario megamorfico sequenziale con $N = 100\,000$: confronto dei livelli JIT (μs/op).

Implementazione	**Tier 0**	**Tier 1**	**Tier 4**
FusedStream (Unsafe)	6251.653 ± 407.145	351.903 ± 1.025	114.744 ± 2.171
FusedStream (Safe)	11451.373 ± 181.944	383.883 ± 3.203	117.412 ± 1.863
Java Manual Loop	10971.256 ± 253.405	384.549 ± 3.535	115.942 ± 1.505
Java Stream	11842.252 ± 216.593	812.151 ± 4.153	474.972 ± 17.787
Scala View	35947.208 ± 543.829	1761.645 ± 24.225	631.320 ± 4.161

A Tier 1 entrambe le versioni fused sono già prossime al ciclo manuale; a Tier 4 il divario tra Safe e Unsafe si riduce ulteriormente.

Il risultato è coerente con la generazione di un ciclo specializzato prima dell'esecuzione. Queste misure confrontano però regimi di compilazione fissati, ciascuno dopo il warm-up previsto da JMH: non misurano direttamente il tempo di avvio né la transizione tra i livelli JIT.

7.2.5 Allocazioni

Le trasformazioni operano su variabili locali e i collector primitivi accumulano direttamente su valori scalari. Restano possibili allocazioni legate alla sorgente, alle funzioni utente, ai risultati raccolti e ai task paralleli; per tipi generici il boxing segue la semantica ordinaria della JVM.

I risultati disponibili riportano tempi di esecuzione, senza misure delle allocazioni o dell'attività del Garbage Collector. Non consentono quindi di quantificare la riduzione delle allocazioni né il suo contributo alle prestazioni osservate.

Capitolo 8

Processo di sviluppo

8.1 Modello di sviluppo

Ho seguito un processo iterativo e incrementale, ispirato alle metodologie agili e adattato a un progetto individuale di durata ridotta. Non ho adottato un framework strutturato come Scrum né formalizzato il lavoro con strumenti come Jira: sprint e backlog sono rimasti impliciti, tenuti a mente anziché tracciati. Il feedback sull'evoluzione del progetto è stato ottenuto principalmente attraverso i test funzionali e i benchmark, utilizzati per guidare le successive iterazioni.

Le caratteristiche del processo sono le seguenti:

- **Prototipo funzionante fin dall'inizio:** la prima versione consisteva in una libreria, semplice ma già completa e ha permesso di verificare presto la fattibilità.
- **Feedback continuo:** i test end-to-end e i primi benchmark hanno portato a rivedere iterativamente sia l'API sia l'implementazione interna.
- **Refactoring incrementale:** l'architettura si è consolidata per passi successivi, mantenendo la libreria funzionante e testata a ogni passaggio.
- **Integrazione continua:** una pipeline di CI eseguita tramite GitHub Actions compila il progetto ed esegue i test a ogni modifica.

Sono presenti macrofasi riconoscibili: verifica della fattibilità, strutturazione della soluzione, revisione dell'architettura e completamento.

8.2 Fasi di sviluppo

Si possono individuare cinque fasi principali:

1. **Prototipazione e verifica di fattibilità.** È stata realizzata una prima implementazione end-to-end, comprendente la DSL, il parsing delle espressioni Scala, una rappresentazione intermedia delle operazioni e la generazione di codice imperativo mediante macro.
2. **Evoluzione della DSL e prime ottimizzazioni.** L'interfaccia pubblica è stata reingegnerizzata introducendo una distinzione più chiara tra sorgenti, operazioni intermedie e operazioni terminali. Sono stati ampliati gli operatori supportati e introdotti controlli sulle pipeline non valide. Le prime valutazioni prestazionali hanno inoltre evidenziato alcuni limiti ed errori di implementazione; sono stati introdotti percorsi specializzati per le sorgenti indicizzabili, la materializzazione in array e le operazioni di somma.

3. **Riorganizzazione della rappresentazione intermedia e delle strategie di esecuzione.** La rappresentazione intermedia interna è stata ristrutturata distinguendo la fase ottenuta dal parsing da quella arricchita con informazioni necessarie alla generazione. I nodi e il *phased indexed container* `Ast` condividono la stessa struttura, con campi aggiuntivi indicizzati per fase tramite match type. Sono state formalizzate le strategie di collect, le proprietà riguardanti la cardinalità, informazioni sull'indizzazione e le condizioni di terminazione anticipata. Questa separazione ha permesso di concentrare l'analisi e le decisioni di ottimizzazione in una fase dedicata, riducendo la complessità del generatore.

4. **Separazione del backend di generazione.** Il backend è stato diviso in due stadi: la costruzione di una rappresentazione delle operazioni secondo un modello imperativo, affidata al `Planner`, e il successivo lowering nell'AST Scala, affidato al `Lowerer`. Questa separazione ha disaccoppiato la definizione del piano di esecuzione dalla costruzione del codice, rendendo la libreria più semplice da manutenere ed estendere.

5. **Introduzione del parallelismo e consolidamento finale.** La libreria è stata estesa per supportare l'esecuzione parallela su sorgenti partizionabili. Sono stati inoltre rafforzati i vincoli di tipo della DSL per escludere a compile time pipeline non supportate. La fase conclusiva ha compreso affinamenti prestazionali e la valutazione comparativa mediante benchmark.

8.3 Gestione del repository

Essendo un progetto individuale, non ho investito particolarmente nelle pratiche di coordinamento del repository. Per il lavoro ordinario ho usato direttamente `main`, senza adottare un processo di sviluppo specifico. Ho definito la formattazione secondo le mie preferenze, fissandole per il codice nella configurazione di Scalafmt, senza dover concordare convenzioni con altri sviluppatori.

8.4 Automazione

Anche l'automazione è rimasta essenziale. Ho introdotto una pipeline di CI per compilare il progetto, eseguire i test e produrre il report di copertura. Non ho dedicato particolare tempo a un processo di release automatizzate. Questa scelta ha permesso di concentrarsi sulla libreria, gestendo queste attività manualmente, dato che il loro costo non sarebbe stato ammortizzato vista la breve durata del progetto.

Capitolo 9

Conclusioni

9.1 Retrospettiva

9.1.1 Difficoltà incontrate

La difficoltà principale è stata la scarsità di documentazione orientata allo sviluppo per le funzionalità avanzate di metaprogrammazione. Non si tratta di un vuoto di letteratura: esiste documentazione accademica sull'argomento, ma i concetti si intrecciano con la gestione dei tipi di Scala, particolarmente complessa, e questo rende difficile passare dalla teoria all'implementazione. Capire come combinare macro e reflection a compile-time ha richiesto più tempo del previsto, soprattutto per ricostruire possibilità e vincoli delle API del compilatore. A questo si è aggiunto il costo delle revisioni architetturali: alcuni confini tra le responsabilità dei componenti sono diventati chiari solo dopo l'implementazione iniziale.

Una delle difficoltà ha riguardato il tooling. Ho trovato il supporto dell'IDE per Scala (VS Code) decisamente meno completo e stabile rispetto a quanto presente per altri linguaggi.

9.2 Limiti e compromessi progettuali

Il compromesso centrale riguarda il rapporto tra le operazioni offerte *out of the box* e la complessità della libreria. Includere nativamente un gran numero di operatori avrebbe reso l'API più completa, ma avrebbe richiesto un trattamento dedicato con un costo, inteso in termini sia di righe di codice che di complessità del codice, più che lineare. Ho quindi scelto di definire un modello che permettesse di rappresentare, tramite estensione esterna, la maggior parte delle operazioni, ottenendo un core più compatto e semplice da verificare. Questa scelta apre inoltre alla possibilità di una libreria esterna dedicata, che raccolga componenti come collector (per esempio `toMap`) e filtri stateful (per esempio `distinct`), validati, senza appesantire la libreria principale.

Queste scelte, insieme al bilanciamento tra espressività dell'API, complessità delle macro e prestazioni del codice generato, hanno comportato i seguenti limiti.

- **Costo della specializzazione**: lo spostamento dell'analisi della pipeline e l'inlining a compile time riduce il lavoro a runtime, ma aumenta il costo di compilazione e la quantità di codice generato, soprattutto con pipeline annidate.
- **Composizione statica**: per rendere analizzabile l'intera pipeline, è necessario che la catena sia espressa al call site in un'unica espressione. Questa scelta limita la possibilità di operazioni come stream intermedi a variabili o di comporli dinamicamente.

9.3 Uso degli LLM

Ho utilizzato gli LLM come supporto per alcune attività circoscritte. In particolare nella stesura della relazione mi sono serviti soprattutto per gli aspetti tipografici e di impaginazione, come la creazione di diagrammi e la rappresentazione delle formule. Li ho inoltre utilizzati per effettuare ricerche di informazioni e come aiuto per velocizzare la scrittura del codice di benchmarking e di test.

Per la generazione dei test e dei benchmark, il contributo degli LLM è stato limitato alla produzione di codice sulla base di specifiche definite preventivamente, comprendenti gli scenari da verificare, le modalità di misurazione e i risultati attesi. Lo sviluppo dell'architettura e delle componenti principali della libreria è stato condotto senza ricorrere a LLM o coding agent.

9.4 Sviluppi futuri

Un possibile sviluppo riguarda l'estensione dell'API con operatori ad oggi non supportati (né esplicitamente né tramite estensione), come `sorted` e la concatenazione di stream. Entrambi escono dal modello di esecuzione attuale basato su un singolo attraversamento.

Una seconda direzione riguarda la generazione del codice. Vorrei introdurre meccanismi di miglioramento delle performance in casi particolari come loop unrolling e vettorizzazione tramite istruzioni SIMD (Vector API).

Andrebbe migliorata inoltre la gestione dell'esecuzione parallela, in particolare per ottimizzare il modello di esecuzione in funzione della dimensione della collezione sorgente.

Infine, vorrei provare a riscrivere la libreria usando il modello di code reflection introdotto da Project Babylon in Java. Questo permetterebbe di ottenere una rappresentazione del codice delle lambda direttamente dal linguaggio, senza dipendere da macro e di applicare il concetto di stream fusion ma lavorando a runtime.

Bibliografia

[1] Duncan Coutts, Roman Leshchinskiy, and Don Stewart. Stream fusion: From lists to streams to nothing at all. *ACM SIGPLAN Notices*, 42(9):315–326, 2007.

[2] Oleg Kiselyov, Aggelos Biboudis, Nick Palladinos, and Yannis Smaragdakis. Stream fusion, to completeness. In *Proceedings of the 44th ACM SIGPLAN Symposium on Principles of Programming Languages*, pages 285–299, 2017.

[3] Dmitry Konstantinov. CASSANDRA-21289: Avoid capturing lambda allocation in UnfilteredSerializer.deserializeRowBody. Apache Cassandra Issue Tracker, 2026. Accessed: 2026-09-27.

[4] Dmitry Konstantinov. CASSANDRA-21588: Stop using java streams in CIDRPermissions.SubsetPermissions#canAccessFrom hot method. Apache Cassandra Issue Tracker, 2026. Accessed: 2026-09-27.

[5] Shayan Najd and Simon Peyton Jones. Trees that grow. *Journal of Universal Computer Science*, 23(1), 2017.

[6] OpenJDK. Java Microbenchmark Harness (JMH). https://github.com/openjdk/jmh, 2024.

[7] John Rose and Project Valhalla Team. The saga of the parametric vm. Technical report, OpenJDK Project Valhalla, 2021. June 2020–April 2021.

[8] Nicolas Stucki, Jonathan Immanuel Brachthäuser, and Martin Odersky. Multi-stage programming with generative and analytical macros. In *Proceedings of the 20th ACM SIGPLAN International Conference on Generative Programming: Concepts and Experiences*, pages 110–122, 2021.

[9] Nicolas Alexander Stucki. *Scalable metaprogramming in scala 3*. PhD thesis, EPFL, 2023.